



University of
Sheffield

Evaluating Carbon-Aware Job Scheduling in High-Performance Computing using Trace-Driven Slurm Simulation

A Stanage Case Study

Yongjiang Liu

Supervisor: Tahsinur R Khan

Co-supervisor: Dr Fred Sonnenwald

A dissertation submitted in fulfilment of the requirements
for the degree of

Master of Science in Advanced Computer Science

School of Computer Science
University of Sheffield

September 2026

Declaration

All sentences or passages quoted in this report from other people's work have been specifically acknowledged by clear cross-referencing to author, work and page(s). Any illustrations that are not the work of the author of this report have been used with the explicit permission of the originator and are specifically acknowledged. I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this project and the degree examination as a whole.

Name: Yongjiang Liu

Signature: Yongjiang Liu (electronic signature)

Date: 16 September 2026

Abstract

Delaying computing jobs until electricity is cleaner may reduce carbon emissions, but also changes competition for shared resources. This dissertation evaluates that trade-off using Stanage job records and trace-driven Slurm simulation. Seven admission policies are compared on five matched dates through 55 completed replays. Carbon estimates combine simulated occupancy, regional carbon intensity and an explicit power scenario.

History-only achieves a mean daily reduction of 0.1021% in the primary load-dependent estimate, with 4.97 minutes of additional reported total waiting per evaluation job. Other results vary by date, and no policy meets every frozen stability condition. Queue timing, model assumptions and concentrated user delay limit the interpretation of small savings. A separate feedback prototype tests delay safeguards. The study provides a completed evaluation of these trade-offs, not measured Stanage electricity savings.

Acknowledgements

I thank my supervisor, Tahsinur R Khan, for his guidance and feedback throughout this project. Our meetings helped me turn a broad interest in HPC carbon monitoring into a focused study of scheduling, waiting time and the limits of carbon estimates. His questions about the data and equations helped me explain more clearly what the results support.

I also thank my co-supervisor, Dr Fred Sonnenwald, and the University of Sheffield Research IT Services team. Fred provided the Stanage job archive and explained the cluster configuration, the available energy information and the practical context of the work. His support made this trace-driven case study possible.

Contents

Acknowledgements	iii
1 Introduction	1
1.1 Background	1
1.2 Problem and motivation	1
1.3 Aim and objectives	2
1.4 Research questions	2
1.5 Contributions and scope	3
1.6 Overview	3
2 Background and Literature Review	4
2.1 Energy, carbon and the measurement boundary	4
2.1.1 Energy efficiency and hardware control	4
2.1.2 Estimators, measurements and validation	4
2.1.3 Choosing a method with limited telemetry	5
2.1.4 Shared energy and the accounting boundary	5
2.1.5 Operational and lifecycle emissions	5
2.2 Temporal flexibility and carbon information	5
2.2.1 Carbon exposure across a job’s runtime	5
2.2.2 Flexibility, deadlines and queue constraints	6
2.2.3 Regional intensity and forecast availability	6
2.2.4 Attributional and marginal emissions	7
2.3 Carbon-aware scheduling and Slurm admission	7
2.4 Waiting budgets and the distribution of service costs	8
2.5 Trace-driven simulation and experimental credibility	9
2.6 Planning and reporting a computational study	10
2.7 Synthesis and research position	10
3 Requirements and Data Analysis	12
3.1 Research design and requirements	12
3.2 Accounting archive and privacy	12
3.3 Exploratory data analysis	16

4	Methodology, Design and Implementation	19
4.1	Experimental design	19
4.1.1	Daily cohorts and carry-in reconstruction	19
4.1.2	Simulated cluster and flexibility	21
4.1.3	Policies and information access	22
4.2	Carbon and service accounting	24
4.2.1	Operational carbon model	24
4.2.2	Time anchoring and service metrics	27
4.3	Comparison and validity	30
4.3.1	Repetition and multi-day interpretation	30
4.3.2	Validity and completion requirements	32
4.4	Pipeline architecture	33
4.5	Trace conversion and event generation	33
4.6	Slurm integration	35
4.7	Admission rules	35
4.7.1	Normal admission and runtime-aware shifting	37
4.7.2	High-carbon admission gate	37
4.7.3	Adaptive Balanced	37
4.7.4	Direct day/night hybrid	40
4.7.5	Oracle load information	40
4.7.6	History-only decisions	40
4.8	Accounting and audit outputs	41
4.9	Multi-day execution	41
4.10	Engineering extension after the frozen comparison	42
4.10.1	Candidate selection and budget accounting	43
4.10.2	Connection to actual simulator events	43
4.10.3	Bounded checks and their scope	45
5	Evaluation and Results	46
5.1	Evidence stages	46
5.2	Historical comparisons and corrections	46
5.2.1	November queue fidelity	46
5.2.2	November carbon and waiting	48
5.2.3	November user burden	50
5.2.4	Why the old December percentage is not retained	50
5.2.5	Historical predictor diagnostics	51
5.3	Completed five-date comparison	51
5.3.1	Direction, scale and the meaning of an average	54
5.3.2	Waiting and concentrated user burden	57
5.3.3	No-action cases and repeat variation	60
5.3.4	Stability under the frozen criteria	62
5.4	Explaining observed carbon increases	62

Contents

5.5	Completion, timing and retained failures	64
5.5.1	Completion and runtime errors	64
5.5.2	Submission residuals after time alignment	64
5.5.3	Negative waiting markers	65
5.6	Bounded feedback-extension results	66
5.6.1	Code behaviour and the functional fixture	66
5.6.2	The paired 64-job trace sample	68
6	Discussion	69
6.1	Answers to the research questions	69
6.1.1	RQ1: Changes in modelled carbon	69
6.1.2	RQ2: Waiting and user burden	69
6.1.3	RQ3: Information and delay safeguards	70
6.1.4	RQ4: Consistency across dates and assumptions	71
6.2	What the carbon model supports	71
6.3	Replay validity and generalisation	72
6.4	What the feedback extension adds	73
6.5	Practical implications and future work	74
7	Conclusion	75
	References	77
	Appendices	81
A	Reproducibility and Evidence Guide	82
B	Complete five-date results	84
C	Acknowledgement of generative AI assistance	89
D	Abbreviations	90

List of Figures

3.1	Annual data processing and daily cohort construction	15
3.2	Monthly workload and completion proportion	16
3.3	Annual runtime and waiting percentiles	17
3.4	Submission patterns by UTC hour and weekday	17
4.1	Paired replay and common accounting design	20
4.2	Prepared five-date workload cohorts	21
4.3	Historical regional carbon-intensity inputs	25
4.4	Occupancy representations and carbon-accounting boundary	26
4.5	Job timeline and waiting-time components	29
4.6	Five-date replay allocation	31
4.7	Implemented simulation and analysis pipeline	34
4.8	B1, B2 and Adaptive release mechanisms	36
4.9	Adaptive Balanced decision flow	39
4.10	Event-driven admission feedback	44
5.1	Source-trace and replay queue fidelity	47
5.2	November carbon and waiting trade-offs	49
5.3	November distribution of policy-delay burden	50
5.4	Later-month predictor coverage	51
5.5	Complete daily primary carbon results	53
5.6	Per-policy daily carbon and paired wait	55
5.7	Complete per-date carbon, mean wait and tail wait	56
5.8	Daily user burden	58
5.9	Direct holds, actual added wait and indirect waiting	59
5.10	Replay variability and model direction sensitivity	61
5.11	Within-window carbon increase decomposition	63
5.12	Queue-position cost in the functional fixture	67

List of Tables

2.1	Related work and its relevance to admission control	11
3.1	Key accounting fields and their interpretation.	13
3.2	Archive processing and data quality	14
4.1	Fixed simulator settings and reconstruction limits.	22
4.2	Frozen policy information and key parameters	23
4.3	Symbols and units in the carbon calculation.	27
4.4	Five-date cohort inventory	31
4.5	Implemented improvements and the evidence required to assess them.	42
5.1	Experimental stages and their role in the dissertation.	47
5.2	November policy comparison	48
5.3	Five-date primary policy summary	52
5.4	Submission-time residuals in existing final replay outputs. Residuals are measured after each replay’s common median-origin alignment; no simulator was rerun.	65
5.5	V2 paired 64-job engineering check. Waiting and runtime errors are in seconds.	68
A.1	Research artefacts and their purposes.	82
B.1	All 35 daily carbon results (2025). $K_C = C_{b,C} - C_{p,C}$ is primary cluster saving (kg CO ₂); $g_C = 1000K_C/N$ (g CO ₂ per evaluation job). R_C, R_R, R_A are load-related saving percentages for capacity-weighted, node-request upper and distinct allocated-node proxies; $R_{C,w}$ is whole-cluster primary saving percentage. Negative means increase. † marks zero-action replay variability. Policy abbreviations follow the results chapter.	85
B.2	All 35 daily wait and burden results (2025). n_D counts directly held jobs; $\bar{D}$ is mean hold. $\overline{\Delta W}$ and $P95(\Delta W)$ summarize paired added total wait; ΔW_0 is the mean among jobs not directly held. U_D, U_+ are the largest user’s shares of total hold and positive added wait. Times are minutes per job in the stated cohort. † marks zero-action replay variability.	87

Chapter 1

Introduction

1.1 Background

High-performance computing supports research beyond the capacity of individual workstations. Shared CPU and GPU resources consume electricity during execution and while idle. Their carbon footprint depends on the equipment included, how energy is estimated or measured, and the electricity carbon intensity. Runtime and resource requests alone cannot determine that footprint [1, 2].

Electricity carbon intensity changes over time. Temporal-shifting studies examine whether jobs that tolerate a later start can run with lower estimated emissions [3, 4]. A long job may still extend into a higher-intensity period, and delayed work may compete with later arrivals. Resource allocation also differs from electrical load. Evaluation must account for these effects and the waiting imposed on users.

The case study is Stanage, the University of Sheffield HPC service, where Slurm manages heterogeneous batch resources [5]. The experiments retain Slurm’s priority, resource-feasibility and job-start decisions. A separate admission layer changes when selected jobs become visible to it, leaving the scheduler in place.

1.2 Problem and motivation

Stanage accounting contains timestamps and resource allocations, but no usable per-job energy measurements. Approximate cage-power figures supplied during supervision support a power scenario, not a calibrated electrical model for each job. Controlled comparisons therefore need explicit assumptions; assigning precise individual footprints would overstate the evidence.

A policy may lower estimated emissions through long delays, concentrate the burden on a few users, or appear favourable under only one utilisation proxy. A carbon percentage alone cannot distinguish these outcomes. Leaving uncertain jobs unchanged may protect service without reducing carbon on that date. The project therefore evaluates the trade-off, rather than selecting the largest isolated saving.

Earlier work established the simulator and candidate rules. The final comparison evaluates seven policies on matched workloads across five dates, with repeated baselines. A bounded engineering extension tests candidate selection, user budgets and queue feedback. Their evaluation states the limits on information access, waiting cost and model certainty.

1.3 Aim and objectives

The aim is to evaluate the carbon and service consequences of carbon-aware job admission through trace-driven Slurm simulation under limited energy telemetry.

The objectives follow the order of the research:

- Audit the accounting archive and describe workload timing, resource use and data-quality limits.
- Reconstruct evaluation workloads, including jobs already running, queued or held at each date boundary.
- Implement representative admission policies and estimate operational carbon within a consistent accounting boundary.
- Compare carbon, service outcomes and user delay on matched workloads, then assess variation across dates, replay repeats and resource-occupancy models.

The outputs include cleaned aggregates, workload manifests, policy decisions, completed replay records and comparative results.

1.4 Research questions

The following questions bring together the issues investigated during the project and define the scope of the dissertation.

1. How do representative carbon-aware admission policies change modelled operational carbon for a matched Slurm workload?
2. What waiting and user-burden costs accompany these changes, and how does the adaptive waiting budget affect that trade-off?
3. How do policies with different runtime/load information and delay safeguards compare, including variants that use estimates from earlier months?
4. How consistent are the carbon and service outcomes across dates, replay variation and occupancy-model assumptions?

These questions need separate answers. Carbon and service outcomes can move in different directions. Historical runtime estimates may still be combined with a carbon series that was unavailable at decision time. Similarly, a replay can complete successfully while providing too little evidence to establish a robust policy benefit.

1.5 Contributions and scope

The implementation converts anonymised accounting into replay inputs and paired carbon/service analyses. It reconstructs running, queued and held carry-in, applies rules with different safeguards and information requirements, and separates workload validity from timing fidelity and policy outcomes. The main comparison covers seven rules on five dates through 55 final replays, showing date-dependent carbon changes, concentrated delay and differences between occupancy models. A later prototype tests candidate feasibility, shared user budgets and cancellation of remaining delay. These small tests establish mechanism behaviour, not another five-date performance result.

The study is retrospective. Finite arrival windows drain without continuous later production arrivals. The simulator executes supplied or capped runtimes; each policy separately limits the runtime information available for its decisions. Carbon uses re-anchored intervals derived from intended release and logged queue wait, with a separate timing audit. Policies share historical regional carbon series. Without archived decision-time forecasts, even the history-only variants cannot establish a complete online carbon-forecast evaluation.

The primary model covers cage-level operational electricity, excluding hardware manufacture, disposal and other embodied emissions; lifecycle accounting needs additional evidence [6]. No production Slurm change, hardware power cap, checkpointing or preemption is deployed. Simulated flexibility is an experimental assumption, not recorded user consent to delayed service.

1.6 Overview

Chapter 2 reviews related work. Chapter 3 describes the requirements and dataset, and Chapter 4 explains the methodology and implementation. Chapter 5 presents the development evidence, five-date comparison and feedback-extension tests. Chapters 6 and 7 discuss the findings, limitations and future work. The appendices provide the evidence guide and complete daily results.

Chapter 2

Background and Literature Review

2.1 Energy, carbon and the measurement boundary

2.1.1 Energy efficiency and hardware control

Energy efficiency reduces electricity consumption; carbon efficiency can also improve by using a lower-carbon supply. HPC surveys distinguish scheduling, application optimisation, frequency scaling and power capping, and discuss the difficulty of modelling heterogeneous processors and accelerators [7, 8]. This study changes a fixed workload’s admission timing. Device power limits remain unchanged, and lower workload energy is not assumed.

Zeus changes batch size and GPU power limits in recurring DNN jobs [9]. It considers energy and training time because lower power may lengthen execution without reducing total energy. Stanage admission instead retains resources and runtimes, changing energy’s timing in the scenario model rather than seeking Zeus’s application-level reduction.

Chase adapts GPU power limits to forecast carbon intensity during execution, changing a running job’s training progress and energy use [10]. Admission shifting changes eligibility instead. Both respond to electricity conditions, but their service costs differ: Slurm waiting and DNN training-time penalties must be interpreted for their respective workloads.

2.1.2 Estimators, measurements and validation

Operational carbon combines energy with an electricity-emissions factor. Green Algorithms estimates energy from runtime, processor characteristics and allocated memory, assuming utilisation and facility overhead [1]. It works without direct measurements, but depends on representative coefficients. Carbontracker samples supported hardware interfaces and combines estimated energy with carbon information [2]. Its boundary matters: GPU monitoring does not automatically include the host CPU, storage, networking or cooling.

Ichnos combines scientific-workflow traces, resource power models and time-varying intensity to estimate carbon after execution [11]. Its validation does not transfer to Stanage without comparable measurements. F-DATA combines Fugaku job, power and energy records,

but cannot replace Stanage telemetry [12]. Here, allocated resources are observed scheduling quantities used as activity proxies, not electrical-utilisation measurements.

A coefficient model permits unit and sensitivity checks; a meter-based estimator also permits comparison with independent electrical measurements. Trace estimators need credible power mappings and precise execution intervals. O’Brien and colleagues explain why predictive models depend on hardware and application characteristics [8]. Matching resource counts therefore establishes workload consistency, not the accuracy of estimated kilowatt-hours.

2.1.3 Choosing a method with limited telemetry

Green Algorithms needs workload descriptors; Carbontracker needs supported device measurements during execution [1, 2]. Stanage provides descriptors and intervals but lacks usable per-job energy, making an explicit power scenario feasible retrospectively. Comparison with a future metered estimate would require matching the measurement scope, sampling interval and idle-energy allocation.

2.1.4 Shared energy and the accounting boundary

Kamiya and Coroamă explain how boundaries, source data and assumptions change data-centre energy estimates [13]. A changed boundary can otherwise be mistaken for improved efficiency. Here, supplied power figures cover cage equipment, and allocation representations share that boundary. The review’s global estimates provide context, not calibration for Stanage coefficients.

2.1.5 Operational and lifecycle emissions

ACT and the HPC lifecycle study by Li and colleagues separate operational emissions from those embodied in manufacture [6, 14]. Moving work on existing hardware does not establish a smaller manufacturing footprint; this evaluation therefore covers operational electricity. Comparisons remain conditional on the power model. Shared assumptions aid comparison, but errors may not cancel when policies change resource placement.

The BLOOM study separates dynamic consumption, idle infrastructure and embodied emissions, each requiring different evidence and allocation assumptions [15]. Scheduling may change execution timing without changing manufacture or idle infrastructure. This dissertation distinguishes load-dependent and whole-cage carbon without transferring BLOOM’s component ratios or training-utilisation assumptions to Stanage.

2.2 Temporal flexibility and carbon information

2.2.1 Carbon exposure across a job’s runtime

Temporal shifting uses the fact that electricity carbon intensity varies during a job’s feasible execution period. Wiesner and colleagues [3] examine how flexibility windows, interruption

assumptions and forecast accuracy affect this opportunity. For a non-preemptive job, the relevant exposure extends across its runtime. A favourable intensity at release can be followed by several unfavourable hours, so selecting the lowest single timestamp is only an approximation to the interval objective. This motivates comparing a simple time-shifting baseline with a policy that evaluates the complete candidate runtime.

2.2.2 Flexibility, deadlines and queue constraints

The workload limits this opportunity. Sukprasert and colleagues [4] analyse ideal and practical shifting bounds across many regions, finding that attainable benefits depend on job duration, deadlines and service constraints. This supports keeping simple baselines in the comparison: a more complex rule should earn that complexity through a better trade-off. A large saving in one setting also need not generalise. In a university cluster, lower-carbon periods must coincide with available capacity and jobs whose users can tolerate a delay.

A flexibility window is different from an application deadline. A maximum admission delay limits the policy’s own action, but cannot limit subsequent queueing or guarantee a completion time. Wiesner and colleagues explicitly consider the conditions under which work may be deferred or interrupted [3]. The policies here use non-preemptive execution and a bounded change in release time. Since their control ends at admission, responsiveness must be measured after replay even when every decision stays within its budget.

An opportunity analysis asks whether the same work would have lower carbon exposure in a later interval. A scheduler evaluation goes further: it asks whether the proposed admission changes still reduce exposure once jobs compete for capacity. Sukprasert and colleagues show how practical restrictions can reduce ideal shifting potential [4]. This project records the first comparison in the decision audit and the second in the completed schedule. The two records help explain small or negative realised changes without assuming that the calculation is wrong.

2.2.3 Regional intensity and forecast availability

NESO provides the regional signal used here. Its documentation describes electricity-generation CO₂ intensity, regional modelling and half-hour updates; the API identifies South Yorkshire as region 5 [16, 17]. A regional value is neither a site electricity meter nor a complete lifecycle greenhouse-gas inventory. An additional distinction concerns information timing. A historical series obtained after an event does not by itself establish which forecast was available when a scheduling decision would have been made. Consequently, retrospective carbon exposure, forecast-informed decisions and empirical forecast-error evaluation require separate descriptions. Synthetic perturbations can test sensitivity, but they cannot substitute for matched archived forecasts and later estimates.

2.2.4 Attributional and marginal emissions

Another distinction concerns the emissions question being asked. Dodge and colleagues distinguish marginal-emissions accounting, which addresses the change associated with an intervention, from attributional accounting based on average electricity intensity [18]. The NESO series used here is a regional generation-intensity signal. Multiplying modelled consumption by that signal attributes an operational footprint to the simulated schedule; it does not measure the additional generator that responds to a particular admission decision. The term “carbon reduction” in the results therefore refers to a paired reduction under the stated attributional model. It should not be read as a validated causal change in grid emissions.

2.3 Carbon-aware scheduling and Slurm admission

GREEN combines carbon objectives with cluster scheduling using ML-job energy information, temporal flexibility and resource allocation, with mechanisms intended to protect completion time [19]. It uses active scheduling rounds, job scaling and checkpoint-based preemption. Stanage’s mixed historical workload lacks equivalent energy profiles and evidence that arbitrary jobs can resize or restart safely. GREEN therefore informs the choice to consider full-execution carbon, resource pressure and waiting protection, but its reported savings are not a benchmark that admission-only control on Stanage should be expected to reproduce.

PCAPS makes another constraint explicit: delaying one task may block dependent tasks and extend the completion of an entire data-processing job. It combines precedence information with a configurable carbon-versus-completion-time objective [20]. This supports transparent trade-off parameters, although the DAG information used by PCAPS is different from the eligibility and resource fields available here. The Stanage study preserves observed eligibility as the boundary after recorded dependencies or holds and does not claim to reconstruct the full application dependency graph.

CATS offers a lighter integration approach: a Python tool that chooses lower-carbon periods for task execution [21]. Its current documentation includes use with `sbatch` [22]. Slurm’s `--begin` option delays allocation eligibility, but does not promise a start at the specified time [23]. The same separation applies here. An external admission layer chooses the release, while Slurm determines the actual start and placement. Carbon must therefore be calculated from the realised simulated schedule, not inferred from the selected release alone.

Deeper changes to Slurm are also possible. Chadha and colleagues [24] extend it to schedule malleable applications that can change their resource allocation at runtime. That work demonstrates a different intervention with different application requirements. Keeping the historical Stanage jobs rigid allows the present experiment to isolate admission timing, while leaving resizing, preemption and hardware power control outside the implemented policy family.

These schedulers control different variables. GREEN changes the amount and timing of ML resources, PCAPS considers precedence-constrained tasks, and CATS provides a

lightweight task-timing interface [19–21]. Their mechanisms are useful references, but their published percentages do not form a common benchmark: jobs, resource models and service constraints differ. Reproducing a reported percentage would require reproducing those conditions. This study tests selected ideas through a shared admission interface so that every policy controls the same part of the scheduling process.

2.4 Waiting budgets and the distribution of service costs

Carbon reduction becomes a service-quality question when it is obtained through delay. Starburst, although designed for hybrid-cloud cost control, offers a useful waiting-budget framework for exposing this trade-off [25]. It assigns different waits to jobs with different resource demands and allows administrators to choose a cost-completion-time balance. The principle can inform a carbon-aware policy without importing its cloud-bursting assumptions. In particular, a bounded allowance can prevent a short job from receiving a delay that is disproportionately large relative to its execution time.

The Adaptive policy uses this idea by adding a runtime-related waiting budget, a minimum expected carbon benefit and an explicit waiting penalty to carbon-sensitive admission. These are project design choices informed by the literature, not a reproduction of Starburst or GREEN. The shared Slurm replay tests whether they improve the trade-off. Even when the admission layer respects its budget, reordering the workload can change the waiting of other jobs.

Aggregate waiting measures can hide important differences. The mean may fall while the tail grows, and an apparently favourable percentile may conceal delay concentrated on one user. The evaluation therefore reports waiting percentiles, slowdown and user-level delay alongside modelled carbon. These measures show the balance among carbon, responsiveness and fairness. A setting can be useful for one objective and costly for another, which a single success or failure label would miss.

Waiting budgets do not make unlike objectives interchangeable. Starburst’s cost-versus-completion-time control motivates explicit service preferences, while PCAPS illustrates why the completion effects of delaying a task depend on surrounding work [20, 25]. A weighted policy score is consequently a decision rule, not a complete evaluation metric. Carbon, total waiting and user burden must still be reported in their original units. This permits a reader to see whether a favourable score was obtained through a material service cost and whether different users experienced that cost unevenly.

User-level carbon attribution introduces a further, separate choice. Wassermann and colleagues distinguish operator-level reporting from user-centred accounting and make the allocation of idle-resource carbon explicit [26]. Their framework connects measurement and resource use to accounting responsibilities. By contrast, the present user metrics allocate deliberate waiting, not a carbon bill. A user who receives much of the policy delay is not necessarily responsible for a corresponding share of cage emissions. Maintaining that distinc-

tion allows burden concentration to be discussed without assigning an unmeasured individual environmental footprint.

2.5 Trace-driven simulation and experimental credibility

The Slurm simulator lineage offers a way to examine scheduling changes without intervening in production. The earlier simulator retains a real Slurm instance with modifications for historical-job replay and faster execution [27]. The later work by Simakov and colleagues [28] emphasises careful transformation of Slurm code and the variability of scheduling realisations. Separately, Jekanovic and colleagues [29] compare a Slurm simulator with a physical installation. Such studies support the evaluation approach while also showing why fidelity must be established for a particular simulator, configuration and workload.

The scheduling configuration can affect outcomes even when workload membership is fixed. Simakov and colleagues study how node sharing and controller configuration change scheduler responsiveness and waiting in Slurm replay [30]. This makes placement and sharing relevant to both service and occupancy analysis. A sum of requested nodes, for example, does not establish the number of distinct hosts active when jobs can share nodes. The current sensitivity calculations retain these different views, while the experiment keeps its chosen Slurm configuration fixed across policies.

Slurm backfill considers requested runtime and resources when deciding whether lower-priority work can proceed without delaying higher-priority reservations [31]. Admission changes can therefore alter interactions among jobs, even when each job’s resources and runtime remain fixed. The current Stange documentation confirms a heterogeneous CPU/GPU environment and Slurm-based submission, but it describes the current service and cannot alone establish the complete 2025 hardware inventory or production configuration [5]. Site history, reservations and administrator actions remain separate evidence requirements.

The comparison here controls job membership, resource requests, runtimes, reconstructed carry-in state and the carbon-accounting horizon, and requires complete execution. These controls make policy outcomes easier to interpret, but cannot establish that simulated waiting matches production waiting. A structurally valid run may also show no carbon benefit. Repeated baselines and checks across dates are especially important when estimated policy differences are small.

Simulator validity has several layers. Input checks establish that the intended job population and resource requests were encoded correctly. Execution checks establish that those jobs were submitted, scheduled and completed. Comparison with a physical system addresses a further question: whether the simulator reproduces the scheduling behaviour of that system under comparable conditions. Jekanovic and colleagues conduct this latter kind of validation for their simulator setting [29]. Passing the first two layers here does not inherit the third from their paper.

The simulator version and configuration also affect the experiment. The earlier Slurm simulator study examines implementation and parameter effects, while the later work identifies

variation between scheduling realisations [27, 28]. Accordingly, this project saves configuration and workload hashes with its results and repeats each date’s baseline. A small carbon difference that changes sign between repeats is weaker evidence than a difference outside the observed repeat range. Chapter 5 makes this comparison using the saved outcomes.

2.6 Planning and reporting a computational study

The evaluation process needs a clear record of what was planned and what was later explored. Souter and colleagues recommend planning analyses to reduce avoidable reruns while retaining useful replication [32]. Here, pilot windows informed policy development before the multi-date campaign was frozen. Recording that plan defines the intended comparison, but does not turn earlier development data into an untouched test set. Chapter 5 therefore separates historical diagnostics, the completed main comparison and the later bounded engineering extension.

Transparent reporting connects a footprint estimate to its calculation method and execution context [32]. For this study, that requires identifying the source workload, policy parameters, power scenario, carbon series and accounting horizon. A second accounting problem would be the footprint of running the research experiments themselves. The simulated Stanage footprint is a counterfactual output and cannot substitute for the actual electricity consumed by the machine hosting the simulator. No estimate of that separate experimental footprint is claimed without the corresponding execution and energy records.

2.7 Synthesis and research position

These studies motivate explicit energy boundaries, full-runtime exposure, capacity awareness, waiting safeguards and controlled replay. This dissertation tests these principles together on a mixed university workload.

Table 2.1 brings these distinctions together. The useful comparison is the intervention each system can make and the information it requires. This makes the scope of an admission-only experiment visible before its results are interpreted.

This dissertation compares normal admission, B1 shifting, B2 gating and Adaptive admission with explicit waiting protection. Oracle and history-based variants add comparisons under different information and safeguards. The completed multi-day evaluation examines how consistently modelled carbon, waiting and user burden change together. Chapter 5 presents the results, including negative and no-action cases.

Table 2.1: Selected related work compared by intervention and information needs. The final column states the design relevance to this project; it is not a comparison of measured savings across systems.

Work	Intervention and information	Relevance to the Stanage study
Let’s Wait Awhile [3]	Execution timing within a flexibility window; job duration and carbon information.	Evaluate exposure over runtime and state the delay allowance and forecast assumptions.
GREEN [19]	ML-cluster scheduling with energy profiles, scaling and checkpoint-based preemption.	Consider carbon and capacity together; application resizing and checkpointing are outside the implemented admission layer.
PCAPS [20]	Carbon-aware scheduling with task-precedence information and a completion-time trade-off.	Make service costs explicit; observed eligibility is available here, but a complete task DAG is not.
Starburst [25]	Job-sensitive waiting budgets for hybrid-cloud cost and completion-time control.	Motivate bounded waiting; cloud migration and cost optimisation are not part of this experiment.
CATS [21, 22]	Lightweight carbon-aware task timing, including documented sbatch integration.	Separate admission eligibility from the actual allocation and start chosen by Slurm.
This study	Fixed-job release-time policies with declared carbon/load information, carry-in and matched Slurm replay.	Compare modelled carbon, waiting and user burden. Multi-day outcomes remain to be completed.

Chapter 3

Requirements and Data Analysis

3.1 Research design and requirements

The study uses controlled trace-driven simulation. On each evaluation date, the baseline and all policy scenarios use identical jobs, requested resources, simulation runtimes, pseudonymous users, flexibility assignments and carry-in work. Only intended release decisions change. The intervention is therefore controlled at the input stage, while the output comparison still has to account for replay variation and time reconstruction. The model recreates selected aspects of Stanage rather than the complete production system.

Four requirements guide the pipeline. Every result must be traceable to its workload and policy parameters. All expected jobs must complete, preventing missing or timed-out work from appearing as a saving. Carbon accounting must include the delayed execution tail and use a common horizon within each comparison. Finally, carbon, service impact and model sensitivity must be reported separately. Together, these requirements address the four research questions in Chapter 1.

Simulator research distinguishes a functioning replay from a demonstrated reproduction of a physical installation [29]. This study checks replay operation directly. Validation against the physical system is limited by the available historical configuration and Stanage telemetry.

The primary simulator is the Slurm simulator associated with Simakov et al. [28]. It allows the project to retain Slurm scheduling logic while replacing wall-clock time and job execution with simulated events. FastSim was investigated during initial development but was not retained in the final evaluation pipeline. This avoids combining results from engines with different input assumptions before either could be validated against the available case-study data.

3.2 Accounting archive and privacy

The supplied 2025 archive contains 4,114,698 raw rows. Deduplication and the retained-year processing produce 4,098,801 jobs associated with 929 anonymised users. These are archive-level counts, not the number of jobs in an individual policy experiment. The preparation

stabilises duplicated column names, parses the pipe-delimited records and interprets timestamps in UTC. Duplicate records for numeric job identifiers across monthly files are resolved by retaining the latest record.

The relevant fields include submission, eligibility, start and completion timestamps; state; partition; requested and allocated CPU, node, memory and GPU resources; and runtime or time-limit fields. Job names, commands, paths and research content are not needed for the comparative analysis. Window-local pseudonyms permit user-burden analysis without exporting raw user identities. Figures and tables in this dissertation use aggregate summaries; the raw archive and private local configuration are not part of the writing package. Table 3.1 uses the export’s field names and Slurm’s accounting definitions [33].

Table 3.1: Key accounting fields and their interpretation.

Field	Meaning and use
JobIDRaw, User	Job identifier for deduplication; user identifier converted to local pseudonyms for burden analysis.
Submit, Eligible	UTC submission and eligibility timestamps. Eligibility marks availability after recorded dependencies or holds.
Start, End	UTC execution timestamps, used to classify running work and examine timing.
ElapsedRaw	Elapsed runtime in seconds; supplies execution duration, subject to the stated replay cap.
TimelimitRaw	Declared time limit in minutes, converted for comparison; not a guaranteed completion deadline.
State, ExitCode	Job outcome and exit status; distinguish completed work from failures.
Partition, QOS	Partition and quality-of-service labels; describe resource access, not recovered historical priority state.
ReqCPUS, AllocCPUS	Requested and allocated CPU counts; allocations do not measure processor activity.
ReqNodes, AllocNodes, NodeList	Requested/allocated node counts and allocated host names; support inventory and overlap analysis.
ReqTRES, AllocTRES	Requested/allocated resource strings, including CPU, memory and GPU quantities. Memory units require parsing.
ReqMem	Requested memory with size and per-CPU/per-node qualifiers; not measured memory consumption.
ConsumedEnergyRaw	Energy field, conventionally joules. This archive has no usable non-zero values for validation.

The original export repeats the `ReqTRES` and `AllocTRES` headers. The reader gives their second occurrences the suffix `__2` and checks for disagreements; they are not additional

resource allocations. Memory suffixes and time units are parsed before comparison. An absent or zero energy field is not evidence that a job consumed no electricity.

Table 3.2: Accounting archive processing and selected quality exceptions. Exception rows overlap; only the first six rows describe the sequential cohort accounting.

Processing step or exception	Count
Raw accounting rows	4,114,698
Duplicate rows removed	14,576
Invalid JobIDRaw excluded	1
Globally unique valid jobs	4,100,121
Outside 2025 or missing Submit	1,320
Retained 2025 submission cohort	4,098,801
Missing Start in retained cohort	92,536
Negative eligible wait	7
Invalid requested GPU counts	2

Submission and eligibility have different meanings. A job can be submitted but unavailable to the scheduler while waiting for a dependency or an administrative hold. The baseline admission event is therefore anchored to **Eligible**, and dependency/hold time is kept as a separate component of user waiting. Treating all submission-to-start time as scheduler delay would assign the policy responsibility for time outside its control.

Figure 3.1 separates the annual descriptive population from the daily replay population. The latter is reconstructed from eligibility and boundary state rather than obtained by simply taking one day of annual submissions.

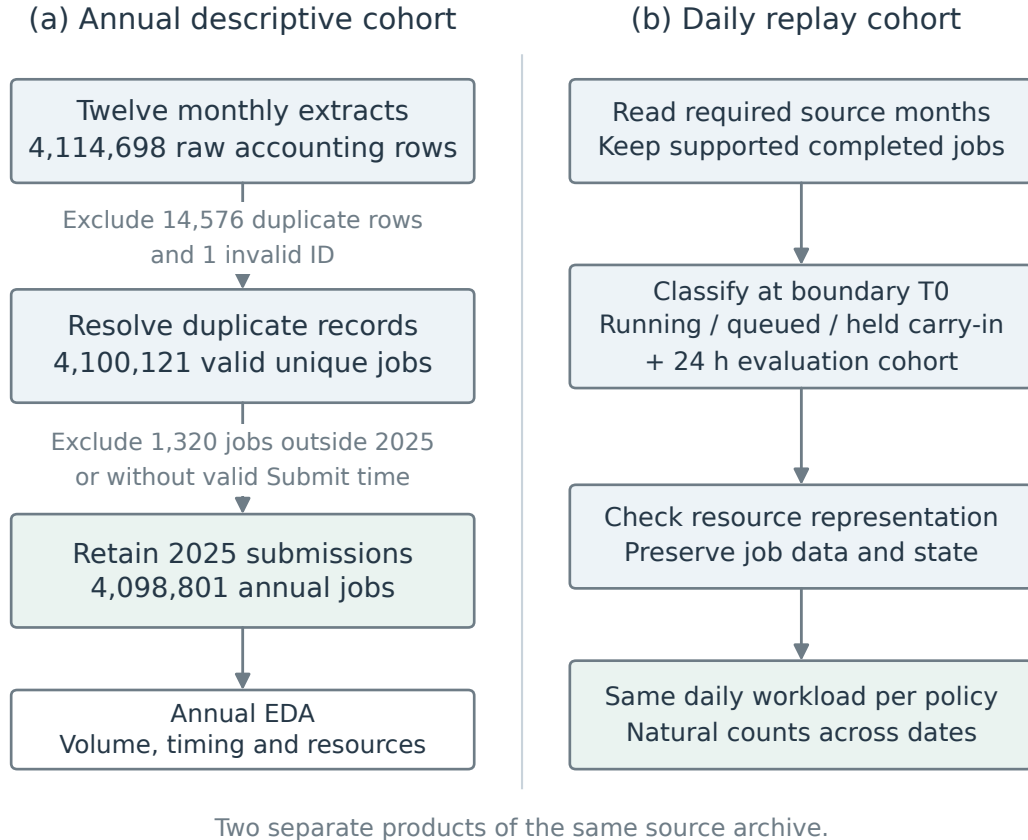


Figure 3.1: Source-data processing for annual description and daily replay. The left panel reports verified archive counts. The right panel describes the separate replay-construction rules and contains no policy outcome. Daily evaluation jobs and carry-in work are defined under the declared eligibility, validity and resource filters.

3.3 Exploratory data analysis

Before selecting evaluation workloads, the EDA examines job volume, states, resource classes, runtime, waiting and arrival patterns. Monthly variation helps identify changes in the archive and provides context for later checks across time. Arrival counts alone cannot indicate electrical utilisation: a short CPU task and a long multi-GPU task occupy very different amounts of resources.

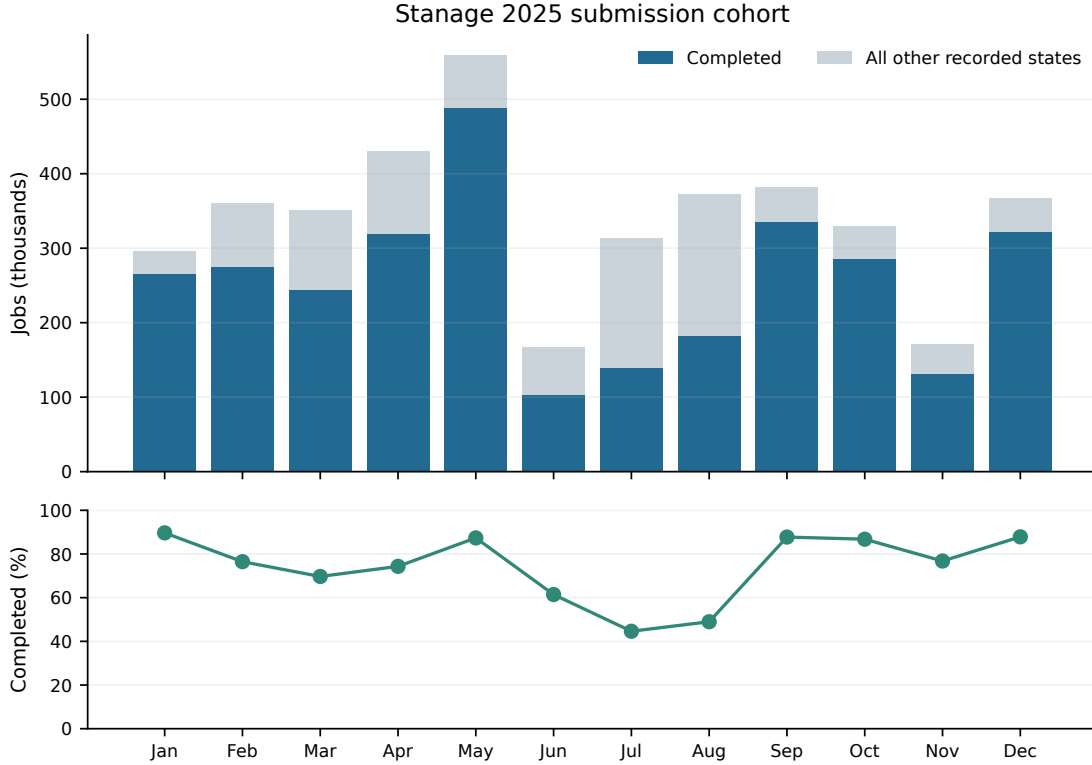


Figure 3.2: Monthly workload and recorded completion proportion in the 2025 submission cohort (4,098,801 retained jobs). Duplicate JobIDRaw records were resolved using the latest source month. All other states include failure, cancellation and non-completed states; the plot does not identify their causes.

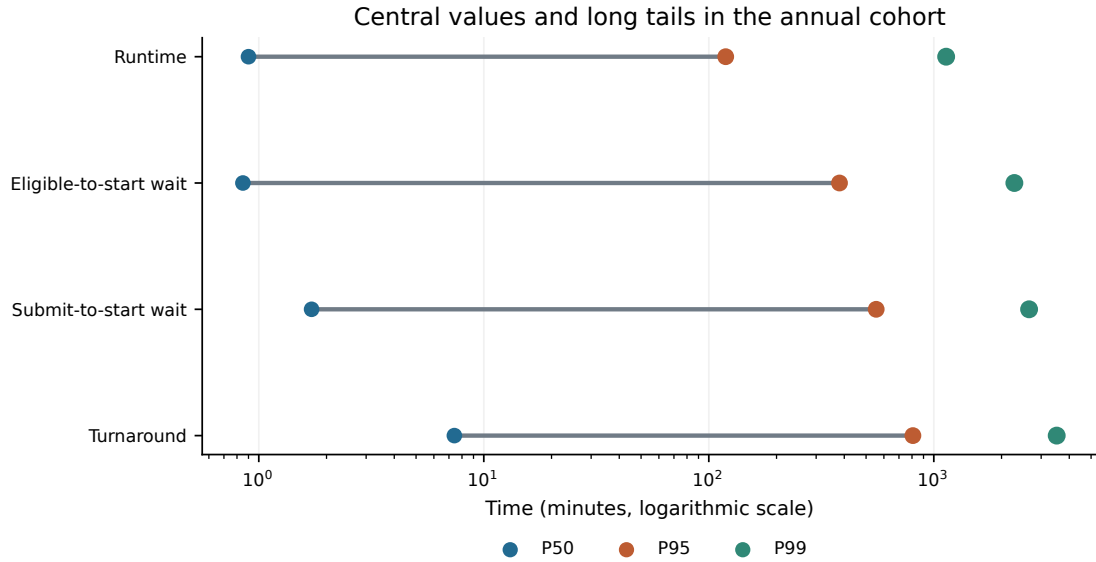


Figure 3.3: Annual timing percentiles from the retained 2025 cohort. Units are minutes, following the CSV unit column despite metric names ending in sec. Valid counts differ: runtime and submit wait 4,006,265; eligible wait 4,006,258; turnaround 4,098,801. Lines connect P50 and P95 only and are not confidence intervals.

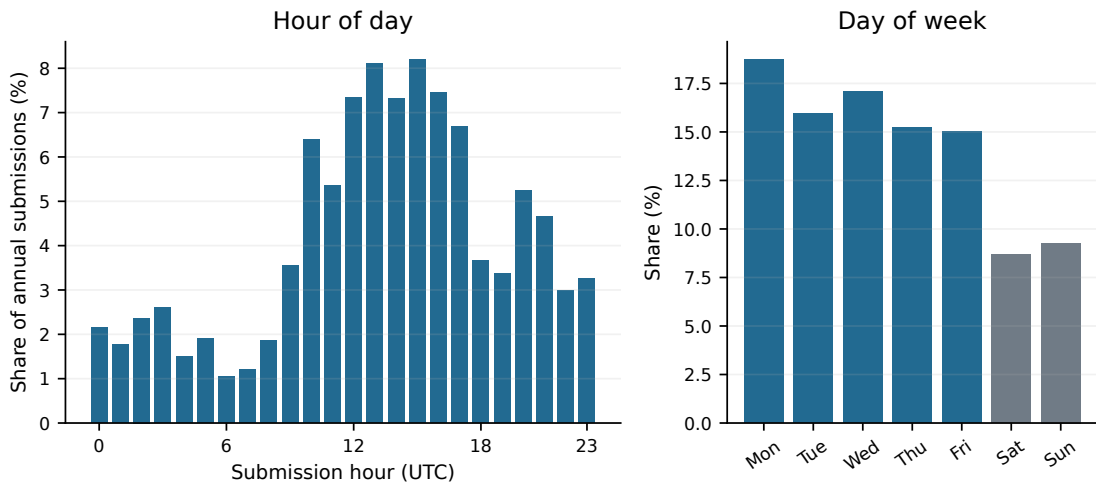


Figure 3.4: Submission-time distribution for 4,098,801 retained 2025 jobs, interpreted in UTC. Percentages are annual totals by clock hour and weekday, not per-day average arrival rates or measures of available scheduling flexibility.

The state audit helps interpret these patterns. In July and August, low completion fractions are associated with failed records concentrated among a small number of pseudonymous users. A holiday explanation would need independent evidence. The archive provides substantial scheduling information, but its lack of usable non-zero per-job energy fields limits validation of electrical estimates.

Earlier June experiments used short pilot windows and a continuous daily window to develop the event pipeline and candidate policies. These remain development evidence because their results informed subsequent choices. The later 12 November comparison was initially frozen, but November then informed the development of the December low-impact rule. Its current role is consequently historical comparison and diagnosis. Describing it as the final untouched test of every later policy would misstate the sequence of work.

Chapter 4

Methodology, Design and Implementation

4.1 Experimental design

4.1.1 Daily cohorts and carry-in reconstruction

The current multi-day design uses 13, 15 and 18 November and 9 and 10 December 2025, each beginning at 00:00 UTC. Dates were selected from calendar and trace availability before inspection of the new policy outcomes. The set includes weekdays, a weekend date and two months. It is a designed case-study sample, not a random sample of the year.

For each date, supported completed jobs eligible in the following 24 hours form the evaluation cohort under the existing partition, resource and validity filters. The runtime cap is 96 hours. One evaluation job was shortened from 847,856 to 345,600 seconds by this cap, identically in every policy for its date. Completion therefore means completion of the configured workloads. A full-day replay covers the eligible population within the declared filters, not every production record. The work continues through queue drainage; it is not stopped after the 24-hour arrival period.

Three carry-in categories represent the pressure already present at the boundary. Running jobs are admitted first with their remaining runtime. Jobs already eligible but still queued are released at the boundary. Earlier-submitted held or dependency-bound jobs enter at their recorded later eligibility. Carry-in classification takes precedence if definitions overlap. Carry-in jobs cannot be delayed by a policy and do not enter the evaluation waiting percentiles, but their resource occupation contributes to energy and carbon accounting.

The replay uses a finite arrival window and then lets the queue drain. Apart from held jobs already in the carry-in set, normal arrivals after the evaluation day are not added. Later execution therefore faces different background competition from an operating cluster. Matching these finite workloads provides a controlled case comparison, but cannot forecast a continuous production workload.

This reconstruction gives each policy the same non-empty starting conditions. Original priority ages, exact historical node placement, reservations and private fair-share state remain unavailable. Replay diagnostics assess the resulting differences from production behaviour. The word “exact” in a script name refers to its construction procedure, not a claim that all production state has been recovered.

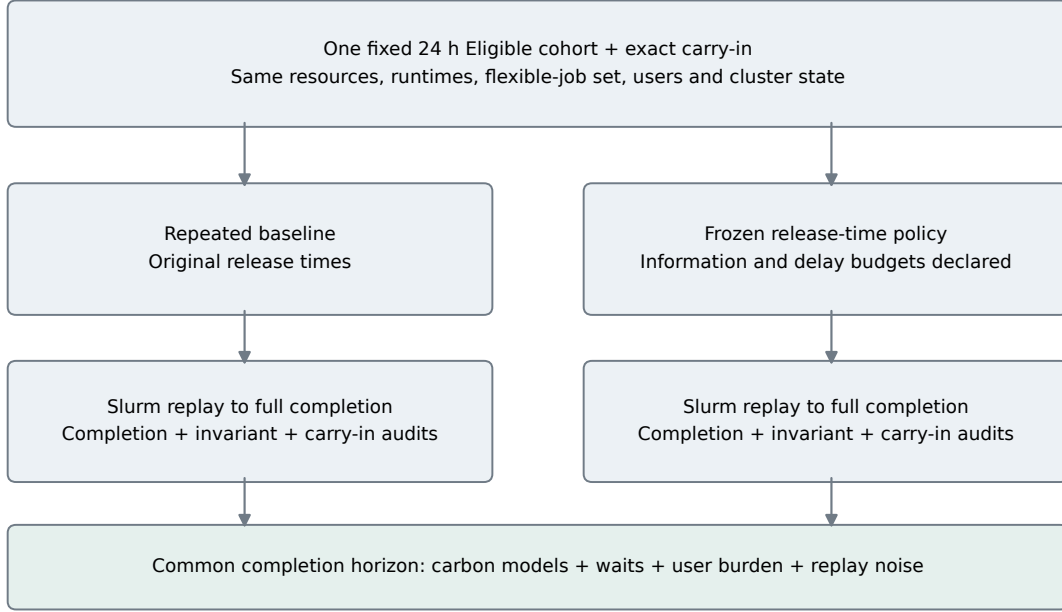


Figure 4.1: Paired replay design. The same date-specific cohort and initial state enter baseline and policy runs, which continue to completion and share a carbon-accounting horizon. The diagram describes the protocol; it does not imply that the five-date campaign has completed.

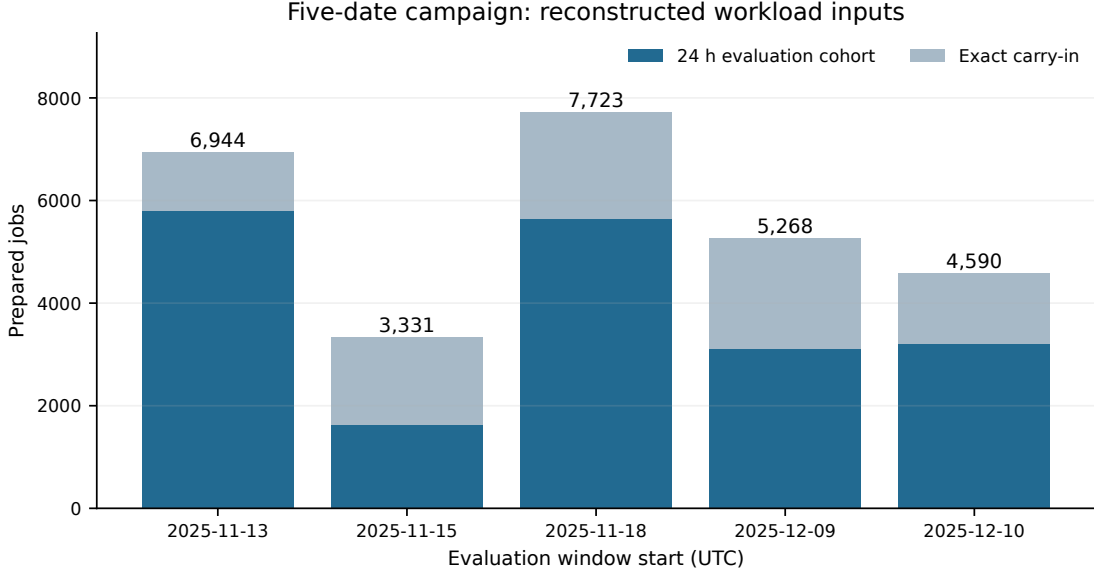


Figure 4.2: Prepared workloads for the five prespecified UTC dates. Each date retains its natural workload size, and all policies within the date receive the same evaluation and carry-in jobs. The figure describes inputs only; it contains no carbon outcome. Cross-date carry-in and completion tails can overlap, so summing these counts does not give unique annual jobs.

4.1.2 Simulated cluster and flexibility

The trace inventory contains 202 worker-node names, grouped into CPU, A100, H100 and H100-NVL classes for simulation. Public Stanage documentation supplies contextual hardware information [5]; mapping it onto the 2025 trace is a modelling step, because a current web page is not a timestamped historical inventory. Node and partition configuration are held fixed within the comparison.

Table 4.1 records the main fixed settings. Holding these settings constant between policies is not the same as reproducing Stanage’s complete production priority configuration. In particular, the replay uses basic priority rather than a reconstructed history of fair-share scores and reservations.

A target probability of 30% assigns flexibility to eligible non-interactive jobs with a fixed seed of 42. The realised fraction can differ from exactly 30% on an individual date. The same assignment is reused for every policy on a date. This provides a controlled intervention population, but it does not identify actual user willingness to wait. Policy findings must therefore be qualified by the assumed flexibility fraction. No job is killed, checkpointed, migrated or given fewer resources to obtain a favourable result.

Temporal-shifting studies treat flexibility and interruption assumptions as determinants of the available opportunity [3, 4]. The fixed assignment here is an experimental control for that opportunity, rather than a forecast of adoption by real users.

Table 4.1: Fixed simulator settings and reconstruction limits.

Setting	Configuration used in the comparison
Scheduling	<code>sched/backfill</code> ; backfill interval 30 seconds.
Resource allocation	<code>select/cons_tres</code> with <code>CR_CPU_Memory</code> .
Priority	<code>priority/basic</code> ; no reconstructed production fair-share history.
Clock	Simulator scale 100, held fixed across the final campaign.
Cluster inventory	202 trace-observed worker names; fixed resource-class and partition mapping.
Initial work	Running, queued and held carry-in; historical placement and priority ages are not fully restored.
Unavailable state	Production reservations, administrator actions and complete accounting associations are not reconstructed.

4.1.3 Policies and information access

The main experiment compares seven representative policies against normal admission. B1 runtime shifting searches within four hours using carbon exposure across the given runtime. B2 delays flexible work when both carbon intensity and projected load are high. Adaptive Balanced combines runtime carbon, waiting and congestion with a runtime-proportional delay budget. A direct hybrid uses B2 during 08:00–18:00 UTC and runtime-aware B1 otherwise. The history-only and low-impact history-only variants estimate decision runtime and load from earlier months. Dynamic Forecast Balanced uses the completed baseline schedule as an oracle diagnostic.

For November, history-based decision tables use January–October; for December, they use January–November. This temporal restriction applies to predictors. The simulator still needs a runtime for execution, and the distinction is preserved in the workload and policy audit fields. The observed-runtime methods and oracle method are informative counterfactuals; they cannot be labelled as tested online predictors.

The representative oracle and history configurations also differ in minimum-saving thresholds, waiting and congestion weights, and carbon-return gates. Any difference between them may therefore reflect both information access and policy safeguards. Isolating the effect of prediction error would require an information-only ablation with those settings held constant.

All methods use the same historical regional carbon curve. The saved series does not record which forecast was available when each production decision would have been made. The label “history-only” therefore applies to runtime and load training information, not to every input in the experiment. Earlier forecast-noise screens test sensitivity to assumed errors; they do not measure forecast accuracy.

Table 4.2 collects the information conditions and key frozen parameters before the implementation details in Chapter 4. In particular, the oracle and history rows expose their different safeguards, so the information comparison can be read with those additional differences in view.

Table 4.2: Information conditions and selected frozen settings for the five-date design. All rows use the same retrospective regional carbon series and 30% target flexibility. R is given observed runtime; $\hat{R}$ is a history estimate. Conditions describe decisions, not measured outcomes.

Policy	Decision information	Delay bound	Key settings
B1 runtime	Given R ; runtime-average intensity	4 h	Minimum average intensity; no load penalty.
B2	Given R for projected node requests; release-time intensity	4 h	Carbon q75; node-request cap = 50% of high-carbon reference peak, minimum 1 node.
Adaptive	Given R ; CPU/GPU class-load projection	$\min(4 \text{ h}, R)$	2% intensity gate; wait/congestion weights 0.10/0.25; carbon q75; class threshold 0.50.
Day/night hybrid	Given R ; information follows selected branch	4 h	B2 at 08:00–18:00 UTC; runtime B1 otherwise.
History-only	Earlier-month runtime, load and queue-wait q75	$\min(4 \text{ h}, \hat{R})$	Capacity/node saving gates 2%/2%; weights 0.10/0.25; class threshold 0.75; return gate 0.05.
Balanced oracle	Given R and completed baseline schedule	$\min(4 \text{ h}, R)$	Capacity/node gates 1%/0.5%; weights 0.03/0.10; class threshold 0.75; return gate 0.02.
Low-impact history	Earlier-month predictions; runtime q25 for delay budget	60 min; also $0.5\hat{R}_{q25}$ and source time limit	Capacity gate 1%; weights 0.01/0.25; class threshold 0.90; return gate 0.001; per user: 60 min and 2 moved jobs.

Weights are wait/congestion coefficients. Return gates use the generator’s carbon-saving proxy per policy-delay hour; they are not validated electrical savings. History and oracle differ in parameters as well as information, so their comparison is not an information-only ablation. B2’s cap is a node-request admission threshold, whereas the other class thresholds enter congestion terms; none is a hardware power limit. Full configuration and prediction safeguards remain in the frozen manifest.

4.2 Carbon and service accounting

4.2.1 Operational carbon model

The primary model translates interval occupancy into a power scenario and integrates it with regional carbon intensity. For class h and half-hour interval k , CPU and GPU fractions are calculated from the occupied resource-seconds divided by configured capacity-seconds. The class index is

$$U_{h,k} = \min\{1, \max(u_{h,k}^{\text{CPU}}, u_{h,k}^{\text{GPU}})\}, \quad U_k = \sum_h \frac{N_h}{202} U_{h,k}. \quad (4.1)$$

The GPU fraction is zero for CPU-only classes. N_h is the number of nodes in the class. This index measures normalised allocation, not processor activity or electrical power. Equal node weights are a transparent simplification rather than a claim that CPU and GPU nodes draw equal power.

Approximate whole-cage values supplied in supervision define

$$P_k = 140 + 55U_k \quad \text{kW}. \quad (4.2)$$

The 140 kW term is the fixed scenario baseline. The supplied 195 kW value describes approximate regular-use power, not a measurement with every resource fully occupied. Setting it at $U_k = 1$ is the project’s normalisation choice. No occupancy observation was supplied alongside that value, so Equation 4.2 is not a curve fitted to two measured power–occupancy points. These inputs cover equipment inside the cage, including compute, storage and networking. Facility overhead outside that boundary has not been quantified, so no additional PUE multiplier is applied.

The general energy-to-carbon conversion follows established accounting practice. Green Algorithms gives its energy estimate in Equation (1) on PDF page 7 and carbon conversion in Equation (3) on PDF page 8 [1]. Carbontracker gives energy and carbon relations in Equations (2)–(3) on PDF page 8 [2]. These sources support the conversion, not the project’s 140/195 kW endpoints or occupancy weights.

For intensity I_k in grams of CO₂ per kWh and duration Δt_k in hours, the load-dependent and whole-cage estimates are

$$C_{\text{load}} = \frac{1}{1000} \sum_k 55U_k \Delta t_k I_k, \quad (4.3)$$

$$C_{\text{cage}} = \frac{1}{1000} \sum_k (140 + 55U_k) \Delta t_k I_k. \quad (4.4)$$

Both return kilograms of CO₂. NESO describes its electricity-intensity signal as generation CO₂ intensity [16, 17]. Legacy project columns containing `co2e` are retained in source provenance, but this dissertation does not turn that signal into a measured full greenhouse-gas lifecycle footprint.

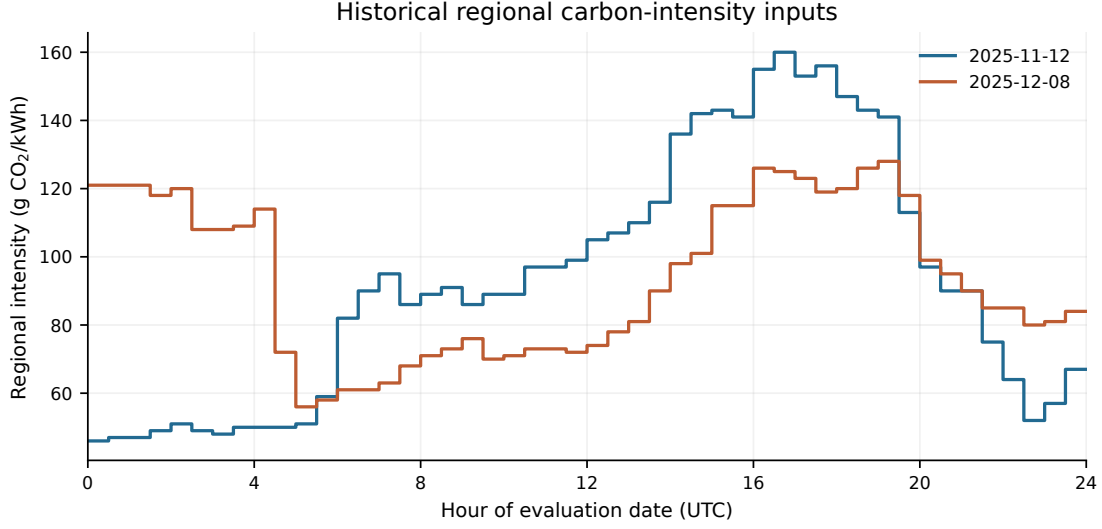


Figure 4.3: NESO regional historical modelled carbon-intensity series used for the November and December case studies, showing the first 24 h of each evaluation window. Each step is 30 minutes. Full emissions accounting includes each common completion tail. The archive has no matched as-issued forecast timestamps and final actual series, so this is a retrospective carbon-signal input, not demonstrated forecast accuracy. The source represents electricity-generation CO₂, not lifecycle CO₂e.

Two sensitivity indices are reported: capped requested-node occupancy and distinct allocated-node occupancy. Requested nodes can overcount shared nodes before capping; distinct allocation merges overlapping intervals on each host. The latter avoids that counting artefact but remains sensitive to simulated node placement.

Slurm node-sharing studies show how placement and sharing configuration affect the relationship between job requests and occupied hosts [30]. The three indices describe different aspects of allocation; they are not independent electrical measurements. Reporting both sensitivity indices alongside the primary index checks whether a favourable result depends on one allocation representation.

This analysis tests sensitivity to a power-model input, reflecting the hardware and work-load dependence discussed in the HPC modelling literature [8]. Without an independent power series for calibration, it cannot provide a confidence interval for measured power.

Figure 4.4 shows how the three occupancy representations enter the same accounting calculation. Keeping the physical boundary and the fixed component visible helps distinguish a load-related saving from a whole-cage percentage.

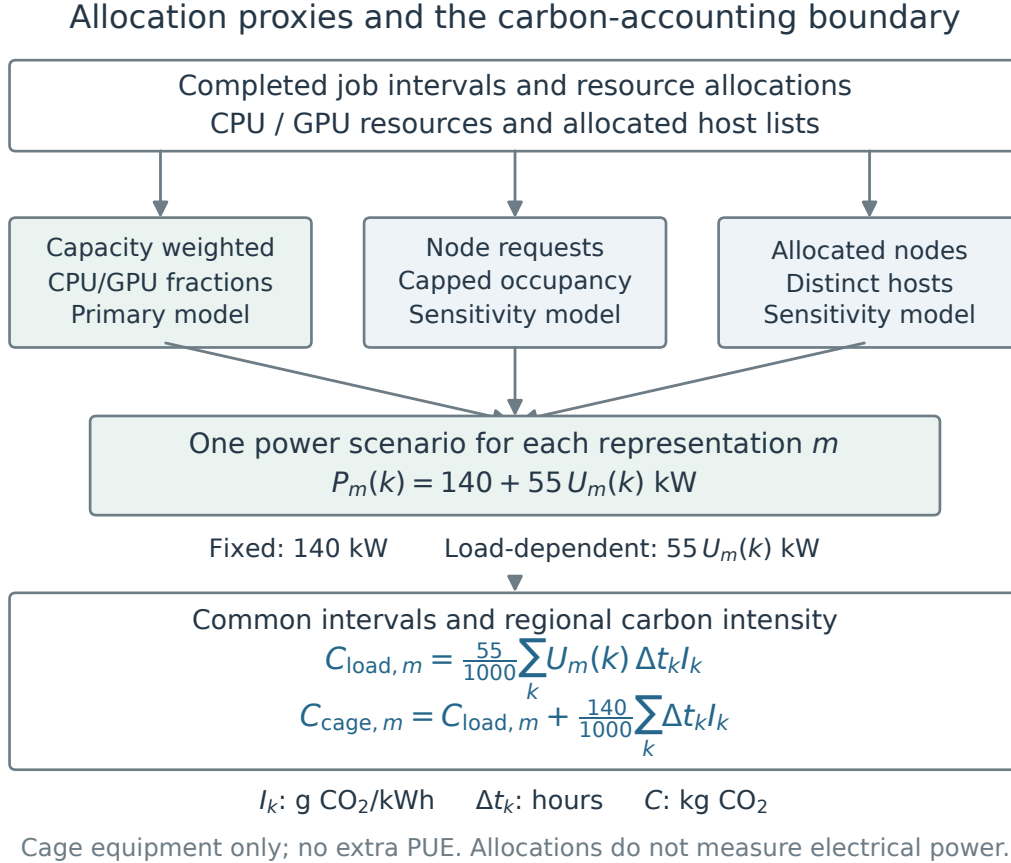


Figure 4.4: Conversion from job allocations to modelled operational CO₂. The primary capacity representation and two node-based sensitivity representations use the same linear cage-power scenario, regional intensity and completion horizon. Fixed and load-dependent components remain separate. Allocation-based occupancy is not measured processor activity or power; the regional signal represents electricity-generation CO₂.

Final accounting uses the same interval for every required replay of a date: from the date boundary to the latest modelled completion, rounded to the carbon interval grid. The accounting start is the intended release plus the non-negative logged scheduler wait; the modelled end adds the configured runtime. These are re-anchored intervals, not the unadjusted logged start and completion times. Section 4.2.2 defines the transformation, and Section 5.5.2 quantifies its submission residuals. The fixed 140 kW integral cancels in a paired absolute carbon difference, but remains in the denominator of a whole-cage percentage. A load-dependent reduction is therefore not the same percentage reduction in total cage emissions.

Table 4.3: Symbols and units in the carbon calculation.

Symbol	Unit	Meaning
h, k	–	Resource class and half-hour accounting interval.
N_h	nodes	Number of simulated nodes in class h ; total 202.
$u_{h,k}^{\text{CPU}}, u_{h,k}^{\text{GPU}}$	–	Occupied resource-seconds divided by class capacity-seconds.
$U_{h,k}, U_k$	–	Class occupancy index and node-weighted cluster index, each bounded by zero and one.
P_k	kW	Scenario power, not a job-level measurement.
Δt_k	h	Duration accounted for in interval k .
I_k	g CO ₂ /kWh	Regional electricity-generation carbon intensity.
$C_{\text{load}}, C_{\text{cage}}$	kg CO ₂	Load-dependent and fixed-plus-load carbon estimates.

4.2.2 Time anchoring and service metrics

For an evaluation job j , let s_j , e_j and r_j be source submission, eligibility and intended policy release, in seconds relative to the date boundary. Simulator logs use a separate virtual-clock origin. Let T_j^{sub} and T_j^{start} denote the raw submission and start log timestamps in seconds. The analyser estimates a common origin for each replay as

$$\theta = \text{median}_i(T_i^{\text{sub}} - r_i), \quad \ell_j = T_j^{\text{sub}} - \theta, \quad b_j = T_j^{\text{start}} - \theta. \quad (4.5)$$

The index i covers jobs with a recorded submission in that replay, including carry-in. Thus ℓ_j and b_j are log-aligned offsets. The submission residual is

$$\epsilon_j = \ell_j - r_j. \quad (4.6)$$

It measures the remaining difference after the common median alignment, not a timezone difference. For evaluation jobs, the reported components of waiting are

$$D_j = r_j - e_j, \quad Q_j = b_j - \ell_j, \quad (4.7)$$

$$W_j = (e_j - s_j) + D_j + Q_j. \quad (4.8)$$

D_j is deliberate policy delay, Q_j logged scheduler waiting and W_j reported total user wait. Hence $W_j = (b_j - s_j) - \epsilon_j$, rather than log-aligned elapsed time from source submission. Moving one job can reorder the queue and change other jobs' waiting.

Carry-in jobs are excluded from these evaluation waiting summaries. Their reconstructed release and eligibility offsets can be set to zero at the boundary even when source eligibility was earlier; the source dependency interval is retained separately. Source and reconstructed eligibility are therefore not interchangeable for carry-in. Carbon accounting includes both cohorts and uses each reconstructed release r_j , the start offset m_j and configured execution duration R_j :

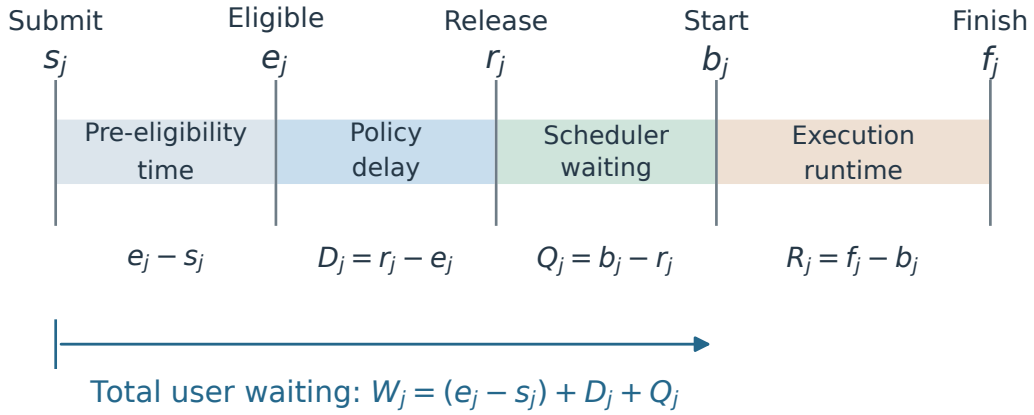
$$m_j = r_j + \max(Q_j, 0), \quad \text{modelled end offset} = m_j + R_j. \quad (4.9)$$

For non-negative Q_j , $m_j - b_j = -\epsilon_j$. Different residuals therefore shift jobs by different amounts; this is not necessarily one harmless translation of the entire schedule. It can alter both carbon-interval exposure and modelled overlap between jobs. The retained carbon and waiting results use this original definition. Its effect relative to a consistently log-aligned alternative has not yet been quantified, and the residual audit is not a replacement carbon calculation.

Negative Q_j values are retained in reported waiting but clipped only in the carbon-start equation. They indicate inconsistent log ordering, not physically negative waiting. Section 5.5.3 reports the affected records; no original waiting value or carbon result has been replaced.

All time quantities here are in seconds. j and i index jobs, θ is the estimated replay origin, $T_j^{\text{sub}}, T_j^{\text{start}}$ are raw log timestamps, ℓ_j, b_j their aligned offsets, and ϵ_j the submission residual. s_j, e_j, r_j describe source or intended events; D_j, Q_j, W_j describe waiting; m_j is the carbon-model start and R_j the configured duration. Figure 4.5 shows the idealised components when the submission residual is zero.

One evaluation job: event order and waiting components



Schematic only: distances do not represent measured durations.

Figure 4.5: Idealised waiting components, assuming intended release and logged Slurm submission coincide. Submission-to-eligibility precedes admission; policy delay ends at release; scheduler waiting ends at start. The reported metric uses logged submission for scheduler waiting when these boundaries differ. The timeline is symbolic and not to scale.

Waiting percentiles, maxima, turnaround and slowdown use the same evaluation cohort. The existing throughput field instead divides all completed replay jobs, including carry-in, by the full replay span; it is reported with that denominator rather than called evaluation-only throughput. The implemented slowdown indicator is $(R_j + W_j) / \max(R_j, 60\text{s})$. Unlike definitions that additionally clamp the ratio to one, this implementation may produce values below one for short jobs; the equation states exactly what is computed. User-level reporting includes cumulative policy delay, the number of affected jobs, maximum burden and the largest user’s share of total policy delay. The count delayed longer than execution runtime is reported separately from compliance with a predictor-based budget.

For a policy p and baseline 0, carbon reduction is

$$S_p = 100 \left(1 - \frac{C_p}{C_0} \right) \%. \quad (4.10)$$

Positive values mean reduction and negative values mean increase. Carbon saved per cumulative policy-wait hour can support efficiency comparisons, but is undefined when no delay is imposed and cannot replace the absolute reduction and burden distributions.

4.3 Comparison and validity

4.3.1 Repetition and multi-day interpretation

Each date received three baseline runs, seven policy runs and one additional Adaptive Balanced run, giving 55 final completed replays. The original plan interleaved baselines with a rotated policy sequence. A recorded execution amendment increased concurrency to five isolated containers across two virtual machines, using the same pinned image and clock scale of 100. The additional environment controls were diagnostic runs outside the 55. Dates, job populations, policy parameters and acceptance rules did not change. Controls on 13 November supported the amended execution on that configuration; they do not establish negligible variation on every date.

The baseline median is the primary paired carbon reference, and the median of two replays represents Adaptive. Repeat ranges show the variation observed within these runs. Each other policy has one replay per date: five different dates do not amount to five repeats of the same scenario. Baseline repeats cannot fully characterise those policies’ run-to-run variability.

The allocation of repeated and single runs is shown in Figure 4.6. All planned cells were completed. Failed or superseded attempts remain in the audit archive and are not counted as extra policy observations.

Table 4.4: Five-date workload inventory before policy comparison. GPU jobs are a subset of each total. Per-date counts may overlap through carry-in and are not unique annual job totals.

Date (UTC)	Evaluation	Carry-in	Total	GPU jobs
2025-11-13	5,806	1,138	6,944	226
2025-11-15	1,629	1,702	3,331	486
2025-11-18	5,646	2,077	7,723	387
2025-12-09	3,113	2,155	5,268	774
2025-12-10	3,210	1,380	4,590	727

Five-date experiment design: planned replays per cell

	Baseline	B1	B2	Adaptive	Hybrid	History	Oracle	Low-impact
13 Nov 2025	3	1	1	2	1	1	1	1
15 Nov 2025	3	1	1	2	1	1	1	1
18 Nov 2025	3	1	1	2	1	1	1	1
09 Dec 2025	3	1	1	2	1	1	1	1
10 Dec 2025	3	1	1	2	1	1	1	1

5 dates × 11 runs per date = 55 planned full replays

3 baseline repetitions; 2 Adaptive repetitions; 1 for each other policy.

Matched workload and initial state within each date; common accounting horizon.

Design only: cells are planned counts, not completed runs or carbon outcomes.

Figure 4.6: Prespecified replay allocation across five dates, completed in the final evidence set. Cell values are repetition counts: three baseline runs, two Adaptive runs and one run for each other policy, totalling 55. The same workload is used within each date, while natural workload sizes differ across dates. The matrix describes allocation rather than carbon outcomes; archived failed attempts and environment controls are not extra policy observations.

The frozen working screen requires five valid dates, material positive primary-model change on at least four, and a worst-day increase no greater than 0.1%. A material positive day must strictly exceed the maximum of 0.05%, the baseline repeat range and the available policy repeat range, and pass the conservative minimum-baseline versus maximum-policy comparison. Both repeat ranges are expressed as percentages of median baseline carbon. The screen also requires positive baseline-carbon-weighted aggregate reduction and positive mean direction when any one date is left out. These are project screening rules, not operational standards or significance tests. Their thresholds were not relaxed after inspecting the results.

Service impact is assessed separately. A lower-impact date requires a P95 increase of at most five minutes, no policy delay longer than configured runtime, at most 60 cumulative delay minutes and two delayed jobs per user, and a largest-user burden share of at most 40%. Carbon stability does not replace these service measures.

Each date retains its natural job count. Equal-day means, medians and worst dates summarise the five cases. A separate pooled percentage uses the sum of baseline carbon as its denominator. Averaging daily P95 values averages percentiles; it does not produce a pooled job-level P95. Carry-in and execution tails may overlap, so the dates are neither independent random observations nor disjoint windows from which to annualise savings.

The user-centred summary first normalises each day’s shared carbon difference by its evaluation-job count, then weights dates equally:

$$g_{p,d} = \frac{1000(C_{0,d} - C_{p,d})}{N_d}, \quad (4.11)$$

$$w_{p,d} = \text{median}_{a,b} \left[\frac{1}{N_d} \sum_{j \in J_d} \left(W_{p,d,j}^{(a)} - W_{0,d,j}^{(b)} \right) \right], \quad (4.12)$$

$$\bar{g}_p = \frac{1}{5} \sum_d g_{p,d}, \quad \bar{w}_p = \frac{1}{5} \sum_d w_{p,d}. \quad (4.13)$$

Here p is a policy, d a date, J_d its evaluation cohort and $N_d = |J_d|$ its size. $C_{0,d}$ and $C_{p,d}$ are median shared-queue carbon estimates in kg CO₂; a and b index policy and baseline repeats. $W_{p,d,j}^{(a)}$ is job j ’s total waiting in seconds. Thus $g_{p,d}$ has units of g CO₂ per evaluation job and $w_{p,d}$ has units of seconds per evaluation job. Carbon includes carry-in and queue-wide effects: $g_{p,d}$ is a scale-normalised cluster difference, not an individual job footprint. Wait differences are calculated for matching job IDs within each replay pair before taking the median pair statistic. Positive-only added wait replaces each difference inside the sum by its maximum with zero, exposing harm that earlier starts elsewhere could otherwise offset.

4.3.2 Validity and completion requirements

Workload identity, resource invariants, carry-in membership, permitted release changes, complete carbon coverage and terminal job completion are checked before a replay enters the comparison. An incomplete date remains visible rather than becoming zero reduction or

disappearing from the five-date denominator. Carbon coverage is provisioned for 14 days; insufficient coverage causes an error rather than interpolation or zero filling.

The frozen manifest and retained deviations support the distinction between planned analysis and later exploration encouraged in research-computing guidance [32]. This is a documented local plan, not an externally registered preregistration. Additional tuning after inspecting the new outcomes would need to be identified as a subsequent development stage.

Chapter 6 assesses the remaining modelling and generalisation limits.

4.4 Pipeline architecture

The pipeline separates data preparation, admission decisions, simulation and evaluation. This allows the inputs to each decision to be inspected and the same completed replay to be analysed under several occupancy models. It produces a workload profile, simulator configuration, event stream, policy decision audit, terminal job log and interval and result summaries. The manifest records parameters and content hashes, since a policy name alone is not enough to identify an experiment.

The implementation began with small simulator examples and then moved to trace-based workloads. Early synthetic jobs helped test events, virtual users, partitions and multi-node requests. These fixtures tested the implementation; they provided no evidence of Stanage carbon reductions. The empirical comparisons in this dissertation use trace-derived workloads, with small fixtures reported separately from completed experimental runs.

Figure 4.7 locates the policy generator relative to the simulator and accounting stages. The separate oracle return path identifies where completed-baseline information enters that diagnostic variant.

4.5 Trace conversion and event generation

The workload generator reads the relevant archive period and constructs a profile with source timing, resource class, requested CPU/GPU/node resources, a pseudonymous user and the execution runtime. The profile also retains source time limits and explicit UTC epoch information. Event offsets are interpreted relative to this epoch, avoiding ambiguous alignment between simulated time, carbon intervals and original timestamps.

An admission decision changes the release offset while leaving resources, identity and runtime unchanged. Events are then regenerated in chronological order and passed to the simulator with `slurm.conf`, `sim.conf`, `gres.conf` and virtual-user definitions. The full profile is checked during comparison, because equal job counts alone can conceal differences in membership or resources.

The carry-in builder identifies the categories defined in Section 4.1.1. Running work has admission precedence and receives the remaining execution duration. Queued work enters at the evaluation boundary. Held work retains its later eligibility. The carry-in audit checks category membership, release offsets, absence of policy delay and completion. It also measures

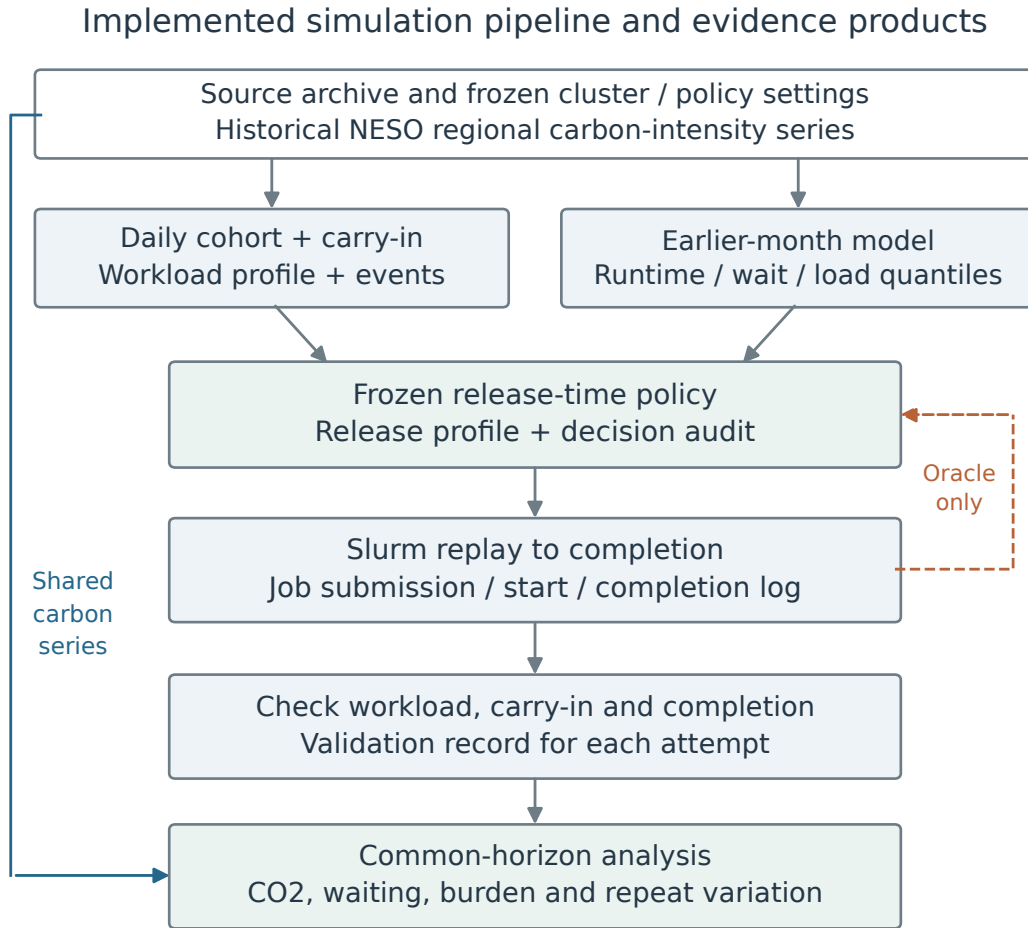


Figure 4.7: Data flow through the implemented simulation pipeline. Earlier-month aggregates supply history predictors. The oracle branch additionally reads a completed baseline schedule, shown by the dashed return path. Policy profiles enter Slurm, while recorded execution and validation outputs support common-horizon accounting. This is an experimental workflow, not a deployed production controller.

running-job startup tolerance, since a simulated release at the boundary does not guarantee an identical historical allocation immediately afterwards.

4.6 Slurm integration

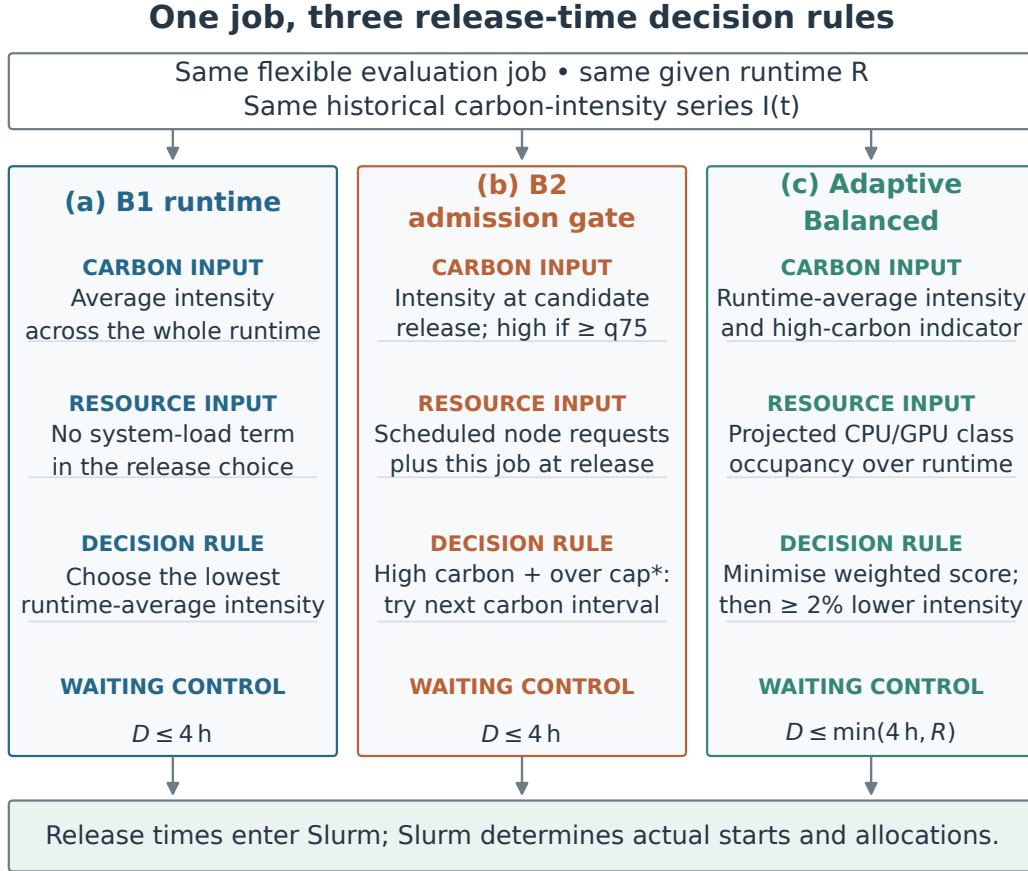
Slurm backfill and resource allocation determine logged starts after admission [31]. All policies use the same simulated priority and resource settings in Table 4.1; they do not reproduce Stanage’s complete production fair-share history. A job admitted during a low-intensity interval can still start later if resources are unavailable. The implementation records planned release and logged start separately. Post-replay carbon accounting then applies the time anchoring in Section 4.2.2; it uses neither the policy’s predicted saving nor unadjusted log timestamps as its result.

Local development included correcting a seconds-to-microseconds conversion in the scheduler source and stabilising the simulator clock scale at 100. Runs are executed with a fixed container image. The runner inspects unique submission, start and completion events and checks that no expected work times out. Some earlier runs terminated abnormally after the final completion event, which motivated checking event completeness in addition to process status. Neither a zero process exit nor a final log line alone establishes a valid result. The relevant run diagnostics are retained with the evidence.

The simulator runs real scheduling code, but replaces scientific application execution with simulated events. It captures scheduling and allocation interactions without measuring application activity, GPU kernels or node electrical demand. Energy analysis therefore remains a separate scenario model.

4.7 Admission rules

The three given-runtime rules offer a direct mechanism comparison in Figure 4.8. They receive the same type of job and historical carbon input, but differ in which release conditions they evaluate and how they limit waiting. The common lower stage is Slurm replay, where the realised start and allocation are determined.



* B2 cap: $\max(1 \text{ node}, 50\% \text{ of reference high-carbon peak node requests})$.

Schematic / frozen configuration. No outcomes. Admission thresholds are not electrical power caps.

Figure 4.8: Comparison of three frozen release rules for the same flexible evaluation job. B1 uses runtime-average carbon intensity; B2 uses instantaneous intensity and node-request concurrency; Adaptive adds waiting and class-load penalties to runtime carbon. Adaptive weights are 0.10 for waiting and 0.25 for congestion, with class threshold 0.50. These methods use given runtime and retrospective carbon information. The diagram is schematic and reports no experiment outcome; oracle and history-based variants have separate information and parameter settings.

4.7.1 Normal admission and runtime-aware shifting

Normal admission releases an evaluation job at its original eligibility. Runtime-aware B1 searches the current release and later half-hour candidates within a four-hour delay limit. For candidate t and decision runtime $\hat{R}_j$, it estimates the time-weighted intensity

$$\bar{I}_j(t) = \frac{1}{\hat{R}_j} \int_t^{t+\hat{R}_j} I(\tau) d\tau. \quad (4.14)$$

Here t and the integration variable τ are time offsets in seconds, $\hat{R}_j$ is the decision runtime in seconds, and $I(\tau)$ and $\bar{I}_j(t)$ are instantaneous and runtime-average intensity in g CO₂/kWh. Using the complete interval avoids treating the first low-intensity half-hour as representative of a long job. B1 uses given observed runtime in the current comparison. It is a controlled full-information reference for the consequences of shifting, rather than evidence that an online runtime predictor is accurate.

The interval objective follows the non-interruptible workload perspective used in temporal-shifting research [3]. The four-hour bound, half-hour candidate grid and observed-runtime input identify this project’s particular comparator; the citation does not supply those parameter values.

4.7.2 High-carbon admission gate

B2 identifies high-carbon intervals using the 75th percentile of the available intensity horizon. Its load threshold is 50% of the peak projected node requests in the reference release profile during high-carbon intervals, with a one-node minimum. This projection uses release offsets and given runtimes, rather than the completed Slurm baseline schedule used by the oracle method. Flexible work may be postponed to subsequent half-hour boundaries, with a four-hour limit. The rule does not replace Slurm’s resource feasibility checks.

Here, “cap” means an admission or concurrency threshold, not a hardware limit measured in kilowatts. The rule can change resource occupancy while leaving the fixed cage-power component unchanged. B2 must therefore be distinguished from the hardware power-management methods described in the literature.

For example, Zeus explicitly changes GPU power limits during DNN training [9]. B2 makes an admission decision and leaves that hardware-control variable untouched, so its threshold must be interpreted in the units of projected node requests.

4.7.3 Adaptive Balanced

Adaptive Balanced evaluates the same carbon opportunity with explicit waiting and congestion costs. Its delay budget is

$$B_j = \min(D_{\max}, \rho \hat{R}_j), \quad (4.15)$$

where $D_{\max} = 4$ hours is the maximum delay and $\rho = 1$ is the dimensionless delay-to-runtime ratio. The calculation uses consistent time units. For each candidate within that budget, the

dimensionless score is

$$F_j(t) = \frac{\bar{I}_j(t)}{\bar{I}_j(e_j)} + \lambda_w \frac{t - e_j}{D_{\max}} + \lambda_c G_j(t). \quad (4.16)$$

The congestion term is implemented as

$$G_j(t) = \begin{cases} \max\{0, V_j^{\text{peak}}(t) - 0.5\}, & I(t) \geq I_{75}, \\ 0, & I(t) < I_{75}. \end{cases} \quad (4.17)$$

$V_j^{\text{peak}}(t)$ is the projected peak normalised resource-class load over the candidate execution interval. $I(t)$ is carbon intensity at the candidate release time, and I_{75} is the 75th-percentile intensity threshold for the available horizon, both in g CO₂/kWh. The load and $G_j(t)$ are dimensionless. Thus the gate tests intensity at release, while the peak load covers the runtime; it does not integrate congestion over every high-carbon subinterval. The frozen weights are $\lambda_w = 0.10$ and $\lambda_c = 0.25$. A move must also reach a minimum expected runtime-average intensity improvement of 2%. The class-load threshold is 50%.

Jobs are processed in eligibility order, with non-flexible and carry-in work keeping their release times. For each flexible job, the generator enumerates candidate times, evaluates interval carbon and projected congestion, and selects the lowest permitted score. An accepted interval updates the load projection used by later decisions. The job audit records the budget, chosen delay and decision reason.

Figure 4.9 gives the implementation order: select the candidate with the smallest score, then apply the minimum carbon-improvement gate. If that selected candidate fails the gate, the job keeps its original release. The expected improvement remains a decision quantity until Slurm replay produces the realised schedule.

The score favours lower carbon while penalising delay and anticipated congestion. Decisions are greedy, so the policy cannot establish a global optimum over all job orderings. It also has the advantage of observed-runtime information. The multi-day comparison tests whether its explicit waiting constraint improves the trade-off relative to B1 and B2.

Starburst’s job-sensitive waiting control motivates the budget, while GREEN informs the joint consideration of carbon and resource pressure [19, 25]. The score and frozen weights are this project’s choices, not formulas taken from those papers. The equations above define the rule tested in the experiments.

Adaptive Balanced: one job in eligibility order

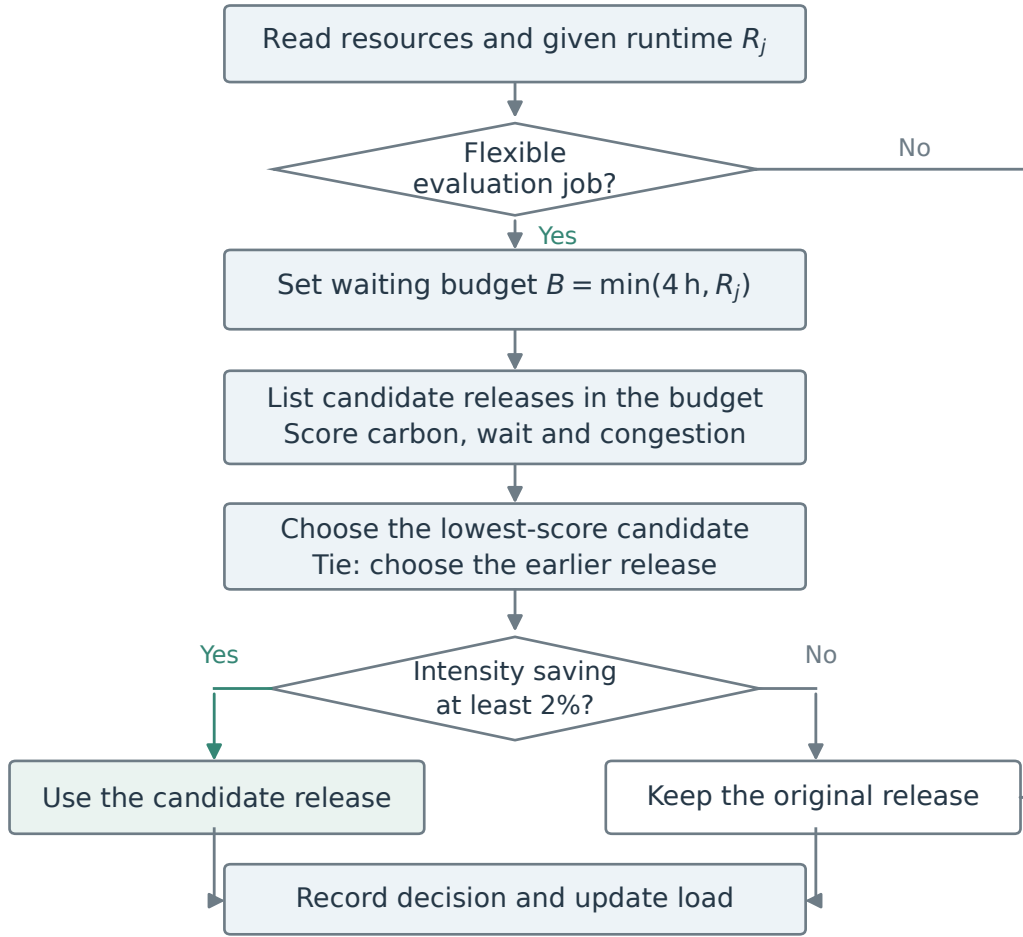


Figure 4.9: Decision sequence for the frozen Adaptive Balanced configuration. A given-runtime budget limits release candidates. The generator first selects the minimum score and then checks the required 2% runtime-average intensity improvement; otherwise it retains the original release. The congestion term is a score penalty, not a measured power cap. Given runtime remains an information advantage in this retrospective comparison.

4.7.4 Direct day/night hybrid

The hybrid follows the job’s UTC eligibility hour. Work eligible from 08:00 to 18:00 uses B2; the remaining hours use runtime-aware B1. Both branches have four-hour limits. The branch is recorded explicitly in the workload profile. This implements the supervisor discussion as a testable comparator. It does not assume that clock time alone consistently identifies low-carbon or low-congestion conditions.

4.7.5 Oracle load information

Dynamic Forecast Balanced reads a completed normal-admission schedule and forms interval-load arrays for CPU, A100, H100, H100-NVL and requested nodes. Before evaluating a job, its current contribution is removed. Candidate contributions are added under the same capping order used by the carbon estimator. An accepted move updates the arrays before later decisions.

The gate requires improvement in both primary capacity and node-request estimates and a sufficient return per unit of waiting. For the current comparison, the frozen minimum predicted savings are 1% and 0.5%, respectively, with a minimum carbon-return parameter of 0.02. Its four-hour/global and one-runtime/job limits remain explicit. These settings identify the saved configuration; they are not claimed to be universally optimal values.

The completed baseline reveals future information about the same workload, so this method is labelled “oracle” in result comparisons. It tests how access to that load information affects release decisions and the carbon and waiting trade-off. The method reads a completed schedule; it neither supplies a measured forecast service nor invokes an AI agent.

4.7.6 History-only decisions

The history model builds runtime, queue-wait and interval-load quantiles from months earlier than the evaluation month. Runtime groups use resource characteristics and deterministic fallback levels where support is insufficient. Earlier development checked expanding history, recent-window history and prior-month calibration. Calibration is assessed on later observations separately from the data used to build the predictor.

The history-only policy obtains its decision runtime and load from frozen quantiles. It uses the dynamic marginal calculation without reading the evaluation job’s completed runtime or the future baseline schedule for the decision. The current configuration uses Q75 runtime, load and queue-wait estimates with a four-hour delay bound. This separation permits comparison of a history-informed policy on the same replayed work. Because the representative oracle and history rules also use different safeguards, observed differences cannot be attributed solely to information uncertainty.

The low-impact variant forms candidates from a five-minute grid, carbon-interval boundaries and the exact budget endpoint, up to 60 minutes. The endpoint can therefore produce a delay shorter than five minutes. It uses Q90 runtime for carbon exposure and a more

conservative delay allowance,

$$B_j = \min\{60 \text{ min}, 0.5\hat{R}_{j,25}, L_j, \text{remaining user delay budget}\}. \quad (4.18)$$

Here $\hat{R}_{j,25}$ is the historical 25th-percentile runtime estimate for job j . L_j is the non-negative declared source time limit; a zero value permits no additional delay. It is an extra duration bound, not a user completion deadline. A user can receive at most 60 minutes of cumulative policy delay and two delayed jobs in the window. Uncertain or unsupported long-GPU cases can retain baseline admission. Carbon, congestion and return thresholds are frozen in the campaign manifest. These constraints limit direct intervention but do not guarantee that Slurm’s subsequent queue response improves aggregate carbon or benefits every user.

4.8 Accounting and audit outputs

The analyser joins source profiles to simulator logs and separates dependency time, deliberate policy delay and scheduler wait. It retains allocated node lists for the distinct-node sensitivity calculation. The carbon calculator computes exact overlaps between re-anchored job intervals and half-hour intensity bins, converts the occupancy index using Equations 4.2–4.4, and produces common-horizon scenario summaries. Exact overlap arithmetic does not establish that the reconstructed intervals have the same timing or resource feasibility as the logged schedule.

Decision-time expectations and realised replay outcomes are stored in separate fields. A predicted saving may disappear when later scheduling moves other jobs, or change sign under another occupancy proxy. Keeping both records helps distinguish correct policy arithmetic from an accurate queue forecast.

Automated checks cover parsing, timestamp conversion, interval overlaps, release constraints, carry-in categories and complete replay denominators. Small test fixtures verify the code and are excluded from the main five-date results. A separately labelled V2 fixture illustrates controller behaviour. The writing package keeps aggregate figure sources and provenance while the full experiment directory retains detailed logs and profiles.

Post-study maintenance adds explicit negative-wait flags and separates job completion, timing validity and process exit. The historical shell wrapper masked the simulator exit code, so a completed replay cannot retrospectively be called a clean process exit. The revised wrapper preserves simulator and Docker exit evidence, with missing evidence marked unknown. These changes improve reporting; they neither repair the unresolved clock behaviour nor alter the 55 frozen replays.

4.9 Multi-day execution

The campaign runner creates a frozen workload and associated policies for each date and saves progress after each complete attempt. The amended dispatcher runs up to five isolated

scenarios concurrently, subject to baseline and oracle dependencies. It records input hashes, the image identifier, configuration, execution backend and audit results. Technical attempts are limited and retained. Resume logic skips only completed runs whose inputs and checks still match, preventing a partial directory from being mistaken for a finished experiment.

Once all relevant runs for a date are available, the summariser uses a shared model-completion horizon and repeated-baseline reference. It reports each date, then the distribution across dates. This ordering prevents a favourable pooled total from obscuring a difficult date. The final evidence set contains all 55 required replays and does not substitute zero for an incomplete attempt.

4.10 Engineering extension after the frozen comparison

The five-date findings prompted a separate implementation revision, called V2. Developed after the main comparison, it left the seven policies and their saved outputs unchanged. Its purpose was to correct specific decision and reporting weaknesses within a three-hour total test budget. It was not another parameter search. Table 4.5 links each change to the problem it addresses.

Table 4.5: Implemented improvements and the evidence required to assess them.

Change	Previous limitation	Implemented response
Feasibility before ranking	The best-scoring candidate could fail a gate while another candidate was feasible.	Filter candidate constraints first; rank eligible alternatives; retain immediate admission as a competitor.
Decision audit	A zero-action result did not always explain the reason.	Record candidate scores, rejection reasons, accepted actions and budget decisions; mark unrecoverable historical reasons as unknown.
Validation by category	Completion could be mistaken for timing or model validity.	Report completion, input identity, initial state, timing, carbon coverage and repeatability separately.
Queue feedback	A static hold did not react to the actual queue.	Reconsider unsubmitted held work on state changes and timer events; release it if the expected benefit disappears.
Shared user budget	Per-job bounds still allowed concentrated cumulative delay.	Count spent and reserved delay against a per-user limit and consider cheaper feasible alternatives.
Failure handling	Controller failure could strand work or reset the test budget.	Fail open by submitting work; persist one total deadline and a closed-experiment record.

4.10.1 Candidate selection and budget accounting

The frozen Adaptive implementation first chooses a score winner and then applies its carbon gate, as Figure 4.9 records. This is retained as the tested original behaviour. The independent V2 copy checks feasibility before ranking candidates in the affected Adaptive, history, forecast and marginal branches. A regression fixture demonstrates a case where the original rule abstains because its score winner fails the improvement gate, while a feasible alternative beats immediate admission. This establishes a code-level difference; it supplies no replacement carbon result for the frozen campaign.

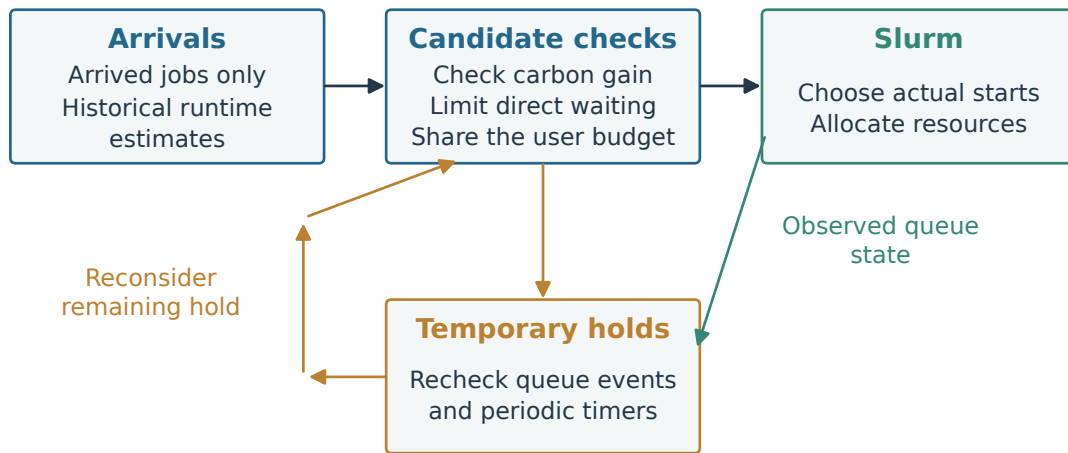
The feedback controller limits each proposed direct hold to the smaller of 30 minutes and half the historical predicted runtime. Each user also has a cumulative 30-minute budget. Spending includes waiting already incurred and reservations for currently held jobs. Same-batch job/time alternatives are ordered by estimated benefit per waiting time. When the best option exceeds the remaining user budget, the controller can consider a cheaper feasible option. Cancelling a hold releases its unused reservation, but does not erase delay already experienced.

4.10.2 Connection to actual simulator events

A C bridge in a separate simulator build intercepts job arrivals before submission to Slurm. It passes arrived-job and submitted-job state to a persistent Python controller. The controller can admit a job immediately or retain it outside Slurm for later review. Queue-state changes and timer checks trigger reconsideration: the timer is 300 seconds for the real-record sample and 60 seconds for the functional fixture. Already submitted or running jobs are neither withdrawn nor interrupted.

The controller estimates remaining work from earlier-month runtime predictions and elapsed running time. It cannot inspect future arrivals and does not use the replay’s configured completion duration as its decision-time predictor. Its simpler resource-work estimate cannot reproduce Slurm priority, fragmentation or backfill reservations exactly; Slurm still determines actual starts and allocations. The retrospective NESO series also remains an input. Evaluating genuinely available decision-time carbon forecasts would require a separate study.

Virtual-clock advancement is paused while the controller processes an exchange. If the controller fails, the bridge records the failure and submits retained and later-arriving work without further carbon holds. The interface has a 4,096-arrival prototype limit. It is an experimental integration rather than a claim of complete production middleware. Figure 4.10 distinguishes the decision loop from the queue it observes.



Benefit lost or controller fails: release held work.

No withdrawal of submitted jobs; no interruption of running jobs.

Figure 4.10: V2 admission feedback, shown schematically. Only arrived jobs and current Slurm state enter the loop. The controller can cancel remaining delay before submission, but cannot undo queue-position changes already caused by the hold. Runtime estimates use earlier-month history; the carbon series is retrospective.

4.10.3 Bounded checks and their scope

The extension was evaluated using unit/regression tests, a paired three-job functional fixture, a paired sample of 64 trace-derived jobs and an injected controller-startup failure. The total recorded elapsed budget was 1,259.72 seconds, about 21 minutes, including build and debugging attempts. No additional training, full-day replay or five-date campaign was started. The input hashes and results of the original comparison were preserved.

The 64-job sample consists of the first eligible records on 18 November that satisfy the CPU-partition, one-node and 60–3,600-second configured-runtime filters. Both runs begin empty and omit the production carry-in workload. Resources and runtimes are unchanged between the paired runs; 17 of the 64 jobs are flexible under the retained selection. The parameters were fixed before this check: a five-minute candidate grid, a 5% minimum predicted runtime-average intensity improvement and the above job/user budgets. Section 5.6 reports the outcomes separately from the main experiment.

Chapter 5

Evaluation and Results

5.1 Evidence stages

The evaluation developed in stages. Synthetic workloads tested event handling and multi-node configuration. June trace windows supported policy development and showed why initial queue pressure and full-runtime carbon exposure matter. A November comparison examined a broader set of rules, followed by a December low-impact check. The main experiment then compared seven representative strategies on the same five dates. Once that campaign was complete, a bounded feedback extension tested specific implementation changes.

Each stage answers a different question. June windows use different cohorts and starting pressure, predictor calibration checks estimates without replaying a counterfactual queue, and November results informed later rules. Table 5.1 records these roles so that earlier work remains visible without its best percentages being presented as final test results.

5.2 Historical comparisons and corrections

5.2.1 November queue fidelity

The 12 November comparison completed twelve scenarios containing 3,900 jobs each: 3,335 evaluation jobs and 565 carry-in jobs. Its source-trace eligible-to-start P95 was 323,424.4 seconds, compared with 53.44 seconds in baseline replay. The queue-fidelity comparison below shows the discrepancy. A complete workload therefore does not establish calibrated production queue timing. Historical priority, fair-share, reservations and administrator state are not fully recovered; simulated waiting is conditional on the reconstruction, not a prediction of actual Stanage service.

Table 5.1: Experimental stages and their role in the dissertation.

Stage	Work completed	Evidential role
Synthetic and June development	Multi-node/partition simulation; trace conversion; initial-state and delay-rule checks.	Development evidence, not the final estimate of savings.
Historical predictors	Earlier-month runtime, queue and load quantiles; later-period calibration diagnostics.	Describes information quality, not measured online carbon-forecast accuracy.
12 November comparison	Twelve scenarios, each with 3,900 jobs.	Historical policy comparison and queue-fidelity diagnostic.
8 December legacy check	Three 5,494-job runs; subsequently audited for initial-state restoration.	Superseded counterfactual evidence because 24 running jobs were not restored at the boundary.
Five-date main campaign	Seven policies per date; three baselines and two Adaptive runs per date; 55 final replays.	Main descriptive comparison, with timing and model limitations reported.
V2 engineering extension	37 code tests, paired three-job and 64-job checks, and a failure check.	Tests implemented mechanisms; not an eighth main-study policy.

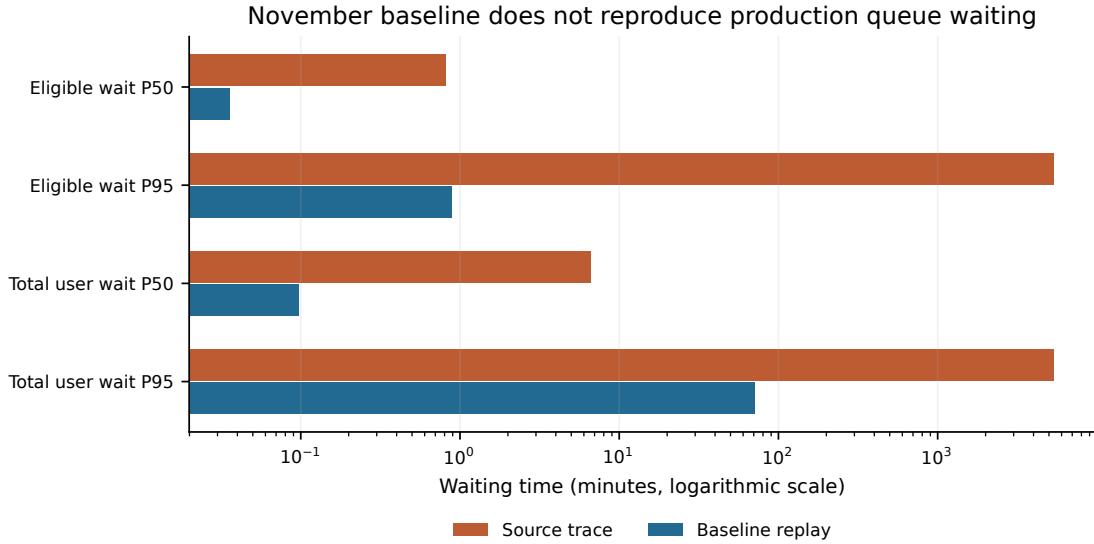


Figure 5.1: Historical preliminary diagnostic. Source-trace and baseline-replay waiting percentiles for the November evaluation cohort. Eligible-to-start P95 was 323,424.4 s in the trace and 53.44 s in replay. The reconstruction does not reproduce production fair-share, reservation and historical-priority state; execution correctness therefore does not establish queue calibration.

5.2.2 November carbon and waiting

The historical comparison shows how the policy design developed. Its paired table and trade-off plot report carbon change alongside tail waiting.

Table 5.2: November single-date diagnostic results. Positive carbon reductions mean lower modelled load-related carbon; a positive P95 change means longer total user waiting. All twelve scenarios completed 3,900 jobs.

Policy	Moved	Delay (h)	Capacity (%)	Node (%)	P95 change (min)
Baseline	0	0.00	0.0000	0.0000	0.00
B1 1 h	407	252.45	0.1638	-0.0634	13.24
B1 2 h	420	583.24	0.4283	-0.1220	43.08
B1 4 h	528	1520.23	0.9797	-0.2353	148.89
B2 Q75 / cap50	373	1114.53	0.6566	-0.2330	149.02
Adaptive balanced	191	484.43	0.6819	-0.0239	70.35
08–18 UTC hybrid	445	1402.52	0.8355	-0.2325	168.47
Strict oracle (no move)	0	0.00	-0.0042	-0.0239	0.20
Balanced oracle (no move)	0	0.00	-0.0053	-0.0243	-0.01
History-only	9	28.75	0.1555	-0.2118	0.01
80% consensus	9	28.75	0.1552	-0.2127	-0.01
Safe rolling (no move)	0	0.00	-0.0059	-0.0243	-0.01

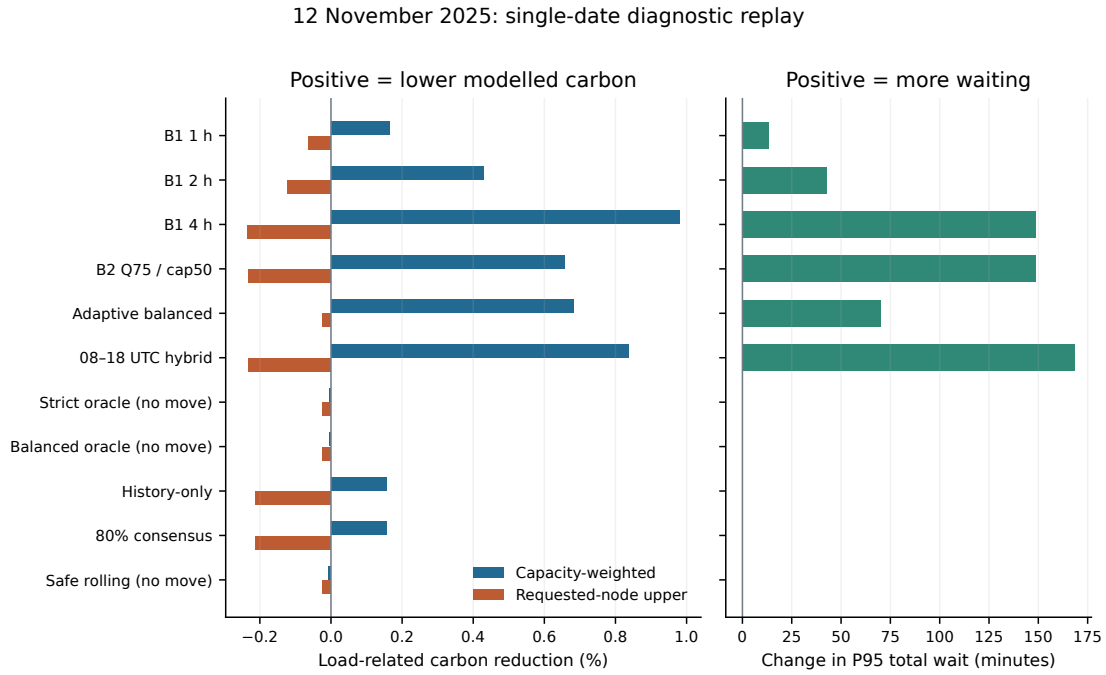


Figure 5.2: Historical preliminary diagnostic. Single-date November replay: carbon-model disagreement and P95 total waiting changes for 3,335 evaluation jobs plus 565 carry-in jobs, with 3,900 completions in every scenario and a common 107.5 h horizon. Positive reduction means (baseline minus policy) divided by baseline. No moving policy improved both proxies. Zero-move oracle and safe differences fall within the observed replay envelope (0.00591% capacity; 0.02432% requested-node) and are not policy effects. November later informed low-impact policy development.

On that date, B1 reduced the primary load-dependent estimate by 0.9797%, added 1,520.23 cumulative job-hours of direct delay and increased P95 total waiting by 148.89 minutes. Adaptive Balanced retained a smaller 0.6819% reduction with 484.43 delay hours and a 70.35-minute P95 increase. Its runtime-related budget therefore offered a less costly trade-off in this historical example. The requested-node changes were instead -0.2353% and -0.0239% , respectively, so neither demonstrates reduction under both representations. The five-date comparison below assesses the same issue on additional matched workloads.

5.2.3 November user burden

Figure 5.3 separates cumulative delay from the carbon and waiting-percentile comparison. Adaptive’s per-job budget limited individual holds, but much of its accumulated delay still fell on one user. These job-hour totals describe delays summed across jobs, not one person’s elapsed waiting time.

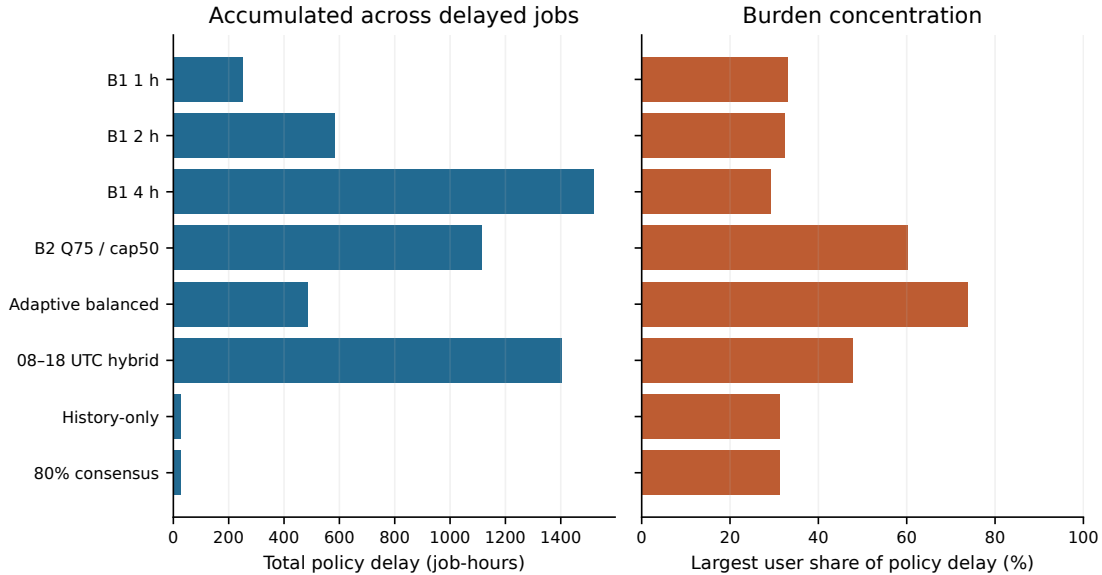


Figure 5.3: Historical preliminary diagnostic. Policy-delay burden in the November single-date replay. Accumulated job-hours sum delays over jobs and are neither experiment duration nor one person’s elapsed waiting time. Adaptive balanced imposed 484.43 job-hours, of which 73.90% fell on one user; satisfying per-job runtime budgets did not establish user-level fairness.

5.2.4 Why the old December percentage is not retained

The earlier 8 December check reported a 0.7179% primary-model increase for Low-impact, alongside lower P95 waiting. A later audit found that 24 jobs that should have been running at the boundary had not been restored; the latest started approximately 12.59 hours later. This changed the initial resource pressure, preventing the pair from identifying the admission

policy’s effect. The original files and percentage remain in the provenance archive, but the chart is excluded from valid carbon comparisons. Excluding the pair corrects the evidence record; it does not establish that Low-impact succeeded. The main five-date campaign passed the corresponding initial-state checks and remains a separate evidence set.

5.2.5 Historical predictor diagnostics

The earlier runtime/load diagnostics below provide context for the history-based policies. A nominal quantile must be checked against later observations rather than assumed reliable from its name. Short jobs, long GPU work and changes between months can have different prediction errors. These checks informed the historical variants, but do not substitute for observing their queue outcomes after admission.

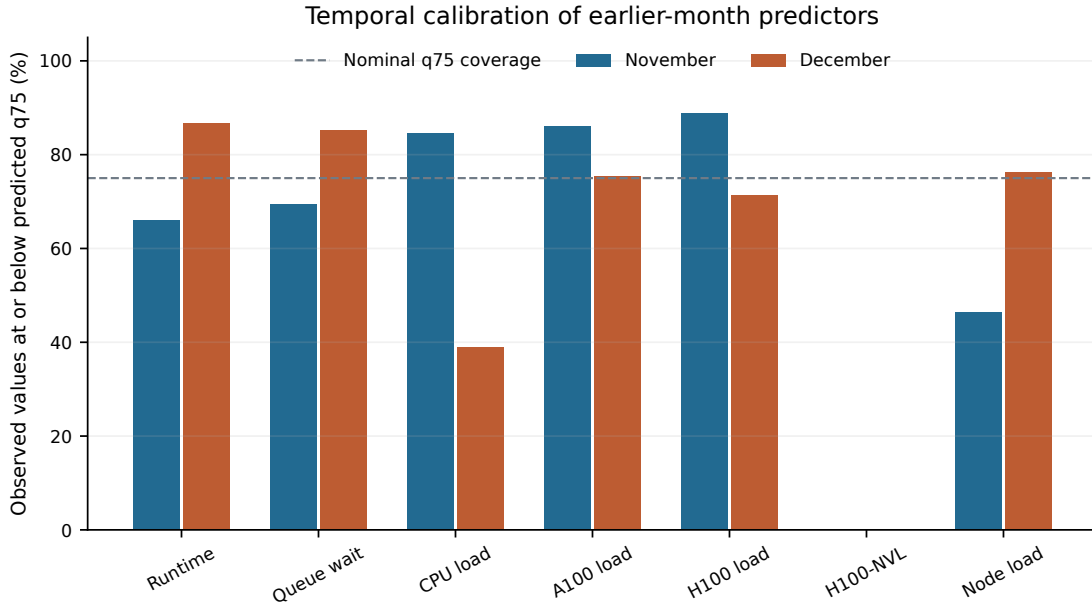


Figure 5.4: Historical preliminary diagnostic. Empirical q75 coverage for history predictors trained on January–October for November and January–November for December. Runtime and queue-wait samples comprise 131,641 and 322,270 valid completed jobs, respectively; load uses 1,440 and 1,488 half-hour intervals. Nominal coverage is 75%. H100-NVL is unsupported by the historical predictor. These are monthly predictor diagnostics, not additional Slurm policy replays or empirical carbon-forecast accuracy.

5.3 Completed five-date comparison

All 55 required final replays completed their expected jobs. The five evaluation populations contain 5,806, 1,629, 5,646, 3,113 and 3,210 jobs, respectively: 19,404 evaluation-job instances in total. Each date also retains its own running, queued and held carry-in work. Different

dates preserve their natural counts; within a date, every policy uses the same cohort, runtime, resources, flexibility assignment and initial-state categories. Thus the study compares the policies on the same five cases, rather than testing each on a different favourable subset.

The results use median repeated-baseline carbon and the median of the two Adaptive carbon values. They retain the original re-anchored accounting intervals and reported waiting definition in Section 4.2.2; the later submission-residual audit does not replace these numbers with log-aligned alternatives. The equal-day normalisations follow Equations 4.11–4.13. The complete daily tables in Appendix B retain all 35 date–policy combinations, including negative and zero-action observations. Positive reduction values mean lower modelled carbon; negative values mean an increase.

Table 5.3: Five-date descriptive primary results. Positive carbon values are savings; negative values are increases. No policy passes the frozen stability screen.

Policy	Mean (%)	Median (%)	Worst (%)	$\bar{g}$ (g/job)	$\overline{\Delta W}$ (min)	A0
B1 runtime	-0.0784	+0.2205	-2.6427	-0.1250	+161.059	0/5
B2	-0.1371	+0.0199	-1.3488	-0.0825	+18.852	3/5
Adaptive	+0.0649	+0.1696	-0.9680	-0.0240	+19.205	0/5
Day/night hybrid	+0.1161	+0.1758	-0.6582	-0.0038	+31.058	0/5
History-only	+0.1021	+0.2418	-0.3028	+0.0123	+4.969	1/5
Balanced oracle	-0.1882	-0.0190	-0.8069	-0.0709	+8.837	5/5
Low-impact history	-0.2612	+0.0156	-1.3723	-0.1039	+17.479	3/5

All rows contain five complete dates. Mean, median and worst summarize daily primary percentage savings; worst is the minimum. $\bar{g}$ averages $1000K_d/N_d$ equally across dates, where $K_d = C_{b,d} - C_{p,d}$ is cluster saving (kg CO₂). $\overline{\Delta W}$ averages daily paired mean added total wait. A0 counts zero-action dates, whose differences reflect replay variability. Different denominators can make mean percentage and mean normalized saving disagree in sign.

All 35 date-policy combinations 2025					
Primary daily carbon saving (%)					
B1 runtime	+1.38	+0.09	+0.55	+0.22	-2.64
B2	+0.57	+0.02 [†]	-0.03	+0.10 [†]	-1.35 [†]
Adaptive	+0.73	+0.00	+0.17	+0.39	-0.97
Day/night hybrid	+0.79	+0.09	+0.18	+0.18	-0.66
History-only	+0.28	+0.01 [†]	+0.24	+0.29	-0.30
Balanced oracle	-0.00 [†]	+0.00 [†]	-0.02 [†]	-0.11 [†]	-0.81 [†]
Low-impact history	+0.02	-0.00 [†]	+0.02	+0.03 [†]	-1.37 [†]
	13 Nov	15 Nov	18 Nov	09 Dec	10 Dec
Teal: saving Amber: increase †: zero action (12 cells)					

Figure 5.5: Primary load-related carbon saving for all 35 date-policy pairs. Cells report $100K/C_b$ (%), where $K = C_b - C_p$ is the common-window cluster saving in kg CO₂. Teal indicates saving and amber increase. Signs are retained when values round to zero; the appendix supplies four decimal places. † marks audited zero release actions, whose nonzero differences reflect replay variability rather than policy effects.

5.3.1 Direction, scale and the meaning of an average

B1 reduced the primary load-dependent estimate on four dates, reaching 1.3835% on its best date. On 10 December it instead increased the estimate by 2.6427%. The equal-day mean reduction was consequently -0.0784% . A four-out-of-five success count would miss the importance of this one larger adverse result.

History-only had a positive equal-day mean of 0.1021% in the primary load-dependent model. Dividing each day's shared carbon difference by that day's evaluation-job count, then averaging the five normalised values, gives about 0.0123 g CO₂ per evaluation job. Mean added total wait was 4.97 minutes per evaluation job. These are population summaries. They do not mean each job saved that amount or waited that long, and the carbon figure is not a measured per-job footprint.

The averaging weights affect the answer. Adaptive had a positive mean daily percentage reduction of 0.0649%, but its mean normalised saving was -0.0240 g per evaluation job and its baseline-carbon-weighted reduction was -0.0126% . Differences in baseline carbon and job count give each date different weights in these summaries, so their signs can differ. All are reported. History-only stayed positive under these conventions, with a pooled load-dependent reduction of 0.0813% and a whole-cage reduction of only 0.00335%. Including the fixed component makes the whole-cage percentage much smaller. Neither percentage is a measured annual saving.

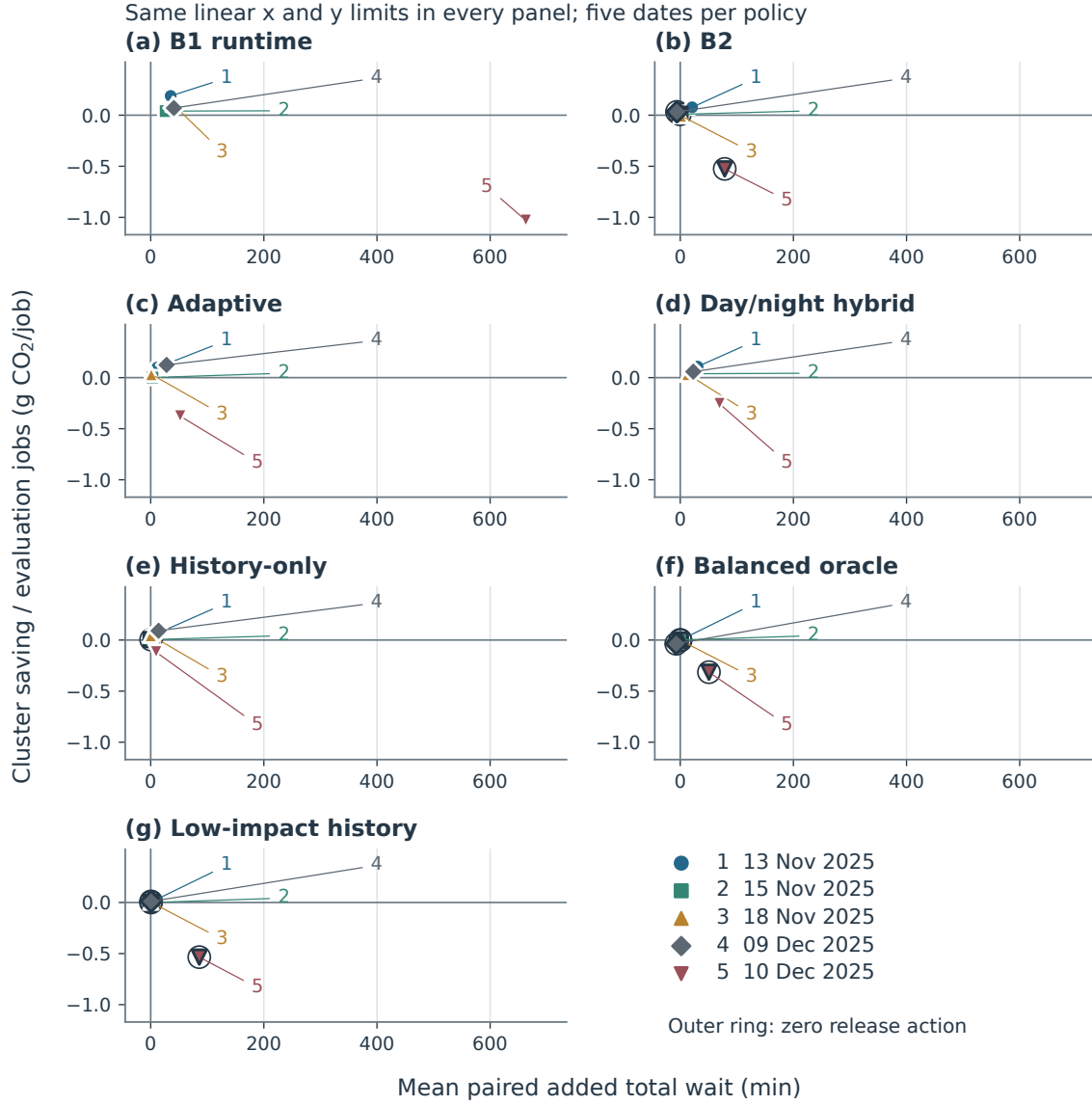


Figure 5.6: Each policy panel retains all five dates, using identical linear axis units and limits. The ordinate is $g = 1000K/N$ (g CO₂ per evaluation job), with cluster saving $K = C_b - C_p$ and evaluation count N ; negative values mean increased carbon. The abscissa is mean paired added total wait in minutes. Numeric labels identify dates. Outer rings mark zero release actions, whose differences reflect replay variability.



Figure 5.7: Complete date-grouped results with common linear limits within each column. Carbon is primary shared-cluster saving $K = C_b - C_p$ (kg CO₂); negative means increase. Wait markers show the mean and P95 of paired total-wait differences in minutes, not differences of separately computed percentiles. AD, HY, HI, OR and LI denote Adaptive, hybrid, history-only, Balanced oracle and low-impact history. † marks zero release actions; their differences reflect replay variability.

5.3.2 Waiting and concentrated user burden

The carbon and waiting plots show each date separately. A direct policy hold accounts for only part of the wait: changes in queue order can affect jobs that were never selected for delay, and replays also vary. Mean waiting differences therefore describe the paired observations, without isolating the causal cost of each selected action.

Across the five dates, B1 deliberately delayed 4,334 evaluation jobs; 3,647 were held longer than their configured runtime. Adaptive delayed 844 jobs, none beyond configured runtime. This verifies Adaptive’s direct-delay bound on the tested jobs. It does not guarantee that scheduler waiting or a user’s cumulative burden is small. History-only delayed 58 jobs, of which 44 exceeded their configured runtime despite meeting predictor-based budgets.

The difference between a population mean and an individual user’s experience is particularly clear for History-only. Its equal-day average direct delay was only 0.523 minutes per evaluation job. On 9 December, however, one pseudonymous user’s 35 jobs accumulated 90.22 job-hours of direct delay and accounted for 86.77% of that day’s policy hold burden. This is a sum over 35 jobs, not a 90-hour hold on one job. The associated overall P95 waiting change was about 21.90 minutes. A small mean does not remove the need for a cumulative user budget.

Direct actions and concentration of positive paired waiting

	(a) Jobs directly held					(b) Largest user's share (%)				
B1 runtime	1026	324	1423	820	741	50.3	47.6	62.1	37.9	48.3
B2	705	0 [†]	260	0 [†]	0 [†]	57.2	31.3 [†]	52.7	26.0 [†]	48.8 [†]
Adaptive	470	73	64	176	61	75.8	61.3	35.3	29.0	53.9
Day/night hybrid	860	252	586	175	81	61.7	39.1	69.3	39.3	39.8
History-only	10	0 [†]	5	41	2	41.9	51.5 [†]	41.8	32.4	34.6
Balanced oracle	0 [†]	0 [†]	0 [†]	0 [†]	0 [†]	62.8 [†]	46.6 [†]	50.6 [†]	21.7 [†]	54.5 [†]
Low-impact history	2	0 [†]	1	0 [†]	0 [†]	65.3	48.5 [†]	52.0	33.7 [†]	52.7 [†]
	11/13	11/15	11/18	12/09	12/10	11/13	11/15	11/18	12/09	12/10

†: zero action; nonzero waiting can still arise from replay variability

Figure 5.8: Direct actions and user-burden concentration for all 35 cases. Panel (a) counts jobs whose releases were intentionally held. Panel (b) reports the largest user's share of positive paired added wait, $100 \max_u B_u / \sum_u B_u$, where $B_u = \sum_{j \in u} \max(\Delta W_j, 0)$; this differs from direct-hold concentration. Panels use separate colour scales. † marks zero actions; apparent waiting burden there reflects replay variability.

Minutes per job in the stated cohort; separate colour scales

(a) Mean direct release hold D: all evaluation jobs

B1 runtime	+36.35	+29.12	+40.31	+41.67	+30.91
B2	+22.02	+0.00 [†]	+1.39	+0.00 [†]	+0.00 [†]
Adaptive	+12.51	+2.00	+1.58	+7.54	+1.15
Day/night hybrid	+31.70	+21.80	+13.62	+8.73	+4.04
History-only	+0.32	+0.00 [†]	+0.16	+2.00	+0.13
Balanced oracle	+0.00 [†]	+0.00 [†]	+0.00 [†]	+0.00 [†]	+0.00 [†]
Low-impact history	+0.02	+0.00 [†]	+0.01	+0.00 [†]	+0.00 [†]
	13 Nov	15 Nov	18 Nov	09 Dec	10 Dec

(b) Mean actual added total wait: all evaluation jobs

B1 runtime	+35.26	+26.45	+39.57	+40.92	+663.10
B2	+21.00	-0.60 [†]	+1.07	-5.81 [†]	+78.60 [†]
Adaptive	+11.62	+3.10	+1.11	+28.05	+52.14
Day/night hybrid	+30.63	+19.36	+12.79	+23.08	+69.43
History-only	+0.40	+0.72 [†]	+0.15	+13.97	+9.61
Balanced oracle	-0.08 [†]	+0.00 [†]	+0.64 [†]	-7.07 [†]	+50.70 [†]
Low-impact history	+0.04	+0.56 [†]	+0.28	+0.66 [†]	+85.85 [†]
	13 Nov	15 Nov	18 Nov	09 Dec	10 Dec

(c) Mean added wait: jobs not directly held

B1 runtime	-1.38	-6.04	-0.23	-6.71	+571.36
B2	-1.22	-0.60 [†]	-0.20	-5.81 [†]	+78.60 [†]
Adaptive	-1.06	-2.42	-0.17	+20.68	+51.75
Day/night hybrid	-1.26	-5.59	-0.31	+22.70	+65.00
History-only	+0.05	+0.72 [†]	+0.05	+10.33	+9.60
Balanced oracle	-0.08 [†]	+0.00 [†]	+0.64 [†]	-7.07 [†]	+50.70 [†]
Low-impact history	+0.02	+0.56 [†]	+0.27	+0.66 [†]	+85.85 [†]
	13 Nov	15 Nov	18 Nov	09 Dec	10 Dec

†: no release action; its apparent spillover is replay variability

Figure 5.9: Direct holds and actual waiting for all 35 cases, in minutes per job. Panels (a–b) average intentional hold D_j and paired added total wait ΔW_j over all evaluation jobs; (c) includes only jobs with $D_j = 0$. Amber means added waiting and teal earlier starts. Different cohort means are not additive components. [†] marks zero release actions, whose apparent indirect waiting is replay variability, not causal spillover.

5.3.3 No-action cases and repeat variation

Low-impact moved only three evaluation jobs across all five dates, with three cumulative hours of direct delay. The Oracle reference moved none. Its access to a completed baseline schedule did not guarantee a feasible candidate under its gates, and it is not a mathematical optimum. Near-zero intervention is therefore reported as abstention rather than as a successful carbon-control result.

The audit identified twelve no-action policy–date combinations whose executable event, user and configuration files were byte-identical to baseline. Their output differences cannot be attributed to intentional delay. On 10 December, the three baseline primary carbon estimates were 125.1411, 125.0978 and 127.1193 kg, a relative range of 1.6153%. The baseline P95-wait range was approximately 107.40 minutes. That day’s no-action B2, Oracle and Low-impact carbon values all fell inside the observed baseline range. Reporting their negative percentages without the no-action marker would give a misleading mechanism explanation.

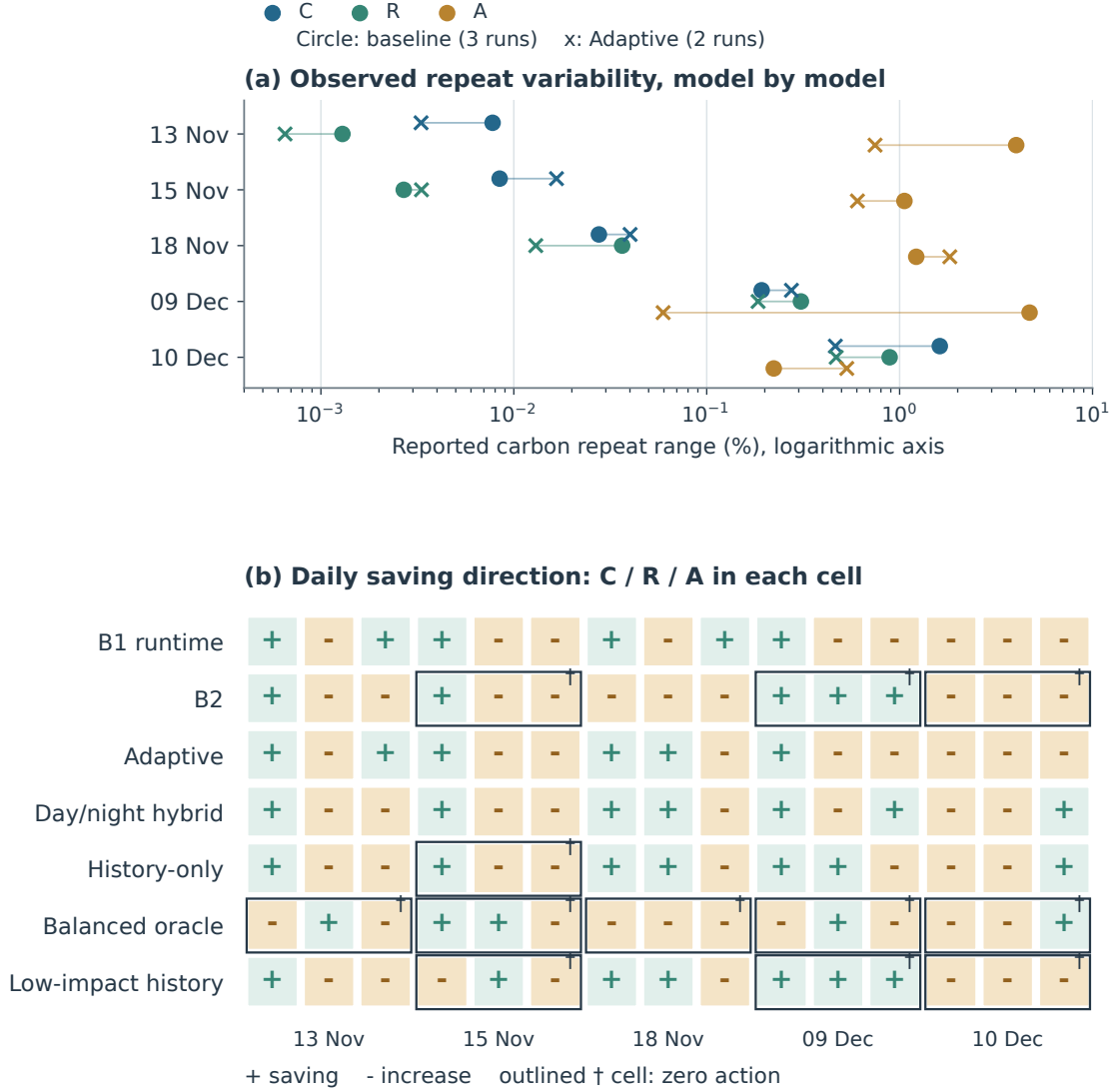


Figure 5.10: (a) Reported within-policy repeat ranges for baseline and Adaptive, with a logarithmic percentage axis. Connecting segments pair descriptive ranges, not confidence intervals. (b) Unrounded saving directions for all 35 cases: C = capacity-weighted, R = node-request upper, A = distinct allocated-node proxies, all load-related. Plus means saving and minus increase; direction alone does not establish robustness. Outlined † cells have zero release actions and reflect replay variability.

5.3.4 Stability under the frozen criteria

No strategy met every frozen carbon-stability condition. B1 exceeded the daily noise and minimum-saving threshold on four dates, but failed on worst-day increase and pooled direction. History-only exceeded that threshold on three dates rather than four, and its worst-day increase was 0.3028%, above the 0.1% tolerance. Its leave-one-date-out mean stayed positive, with a minimum of 0.0562%, which is a useful descriptive indication but does not override the failed conditions.

The seven configurations consequently have mixed or no-action outcomes rather than a single robust ordering. Three occupancy models also disagree frequently: only B2 and Low-impact each had one date positive in all three models; the remaining rules had none. This is sensitivity to allocation representations, not a majority vote among independent meters. The mean, median, spread and worst date describe the cases; with five selected dates and unequal repeats, they are not a basis for an unsupported significance claim.

5.4 Explaining observed carbon increases

Before explaining a carbon increase, it is necessary to check whether the policy changed any admissions, whether the initial state and workload membership matched, and how the difference compares with observed replay variation. An interval-level decomposition can then show where the accounting change occurred.

For a policy-minus-baseline difference, write the energy change in interval k as ΔE_k and let $\bar{I}$ be the common window's duration-weighted mean intensity. The following identity separates total-energy and timing contributions:

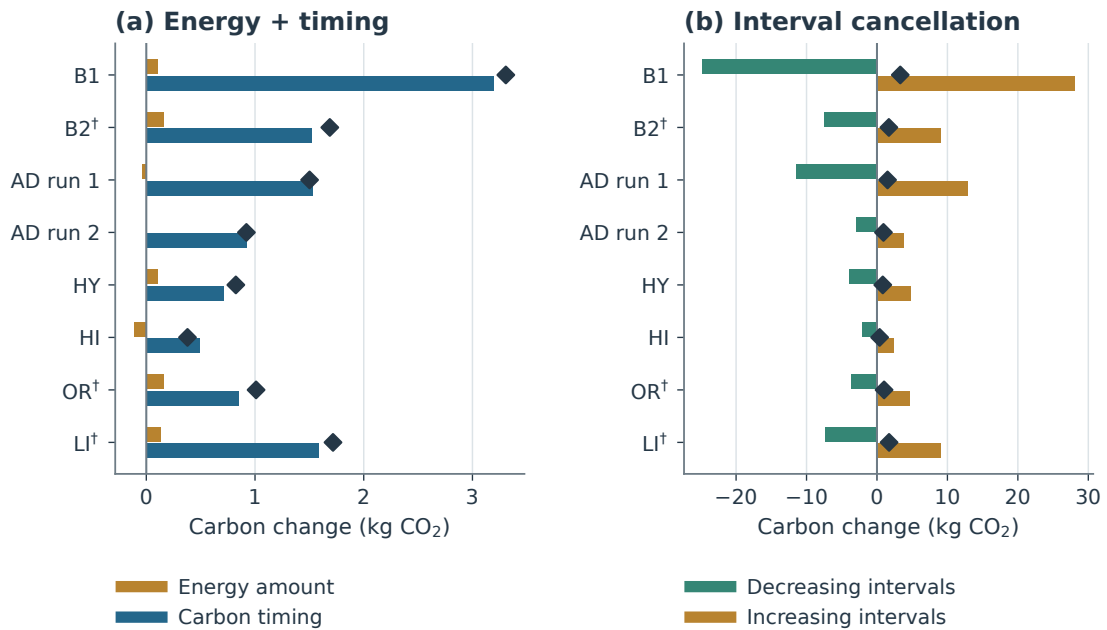
$$\Delta C = \frac{\bar{I} \Delta E}{1000} + \frac{1}{1000} \sum_k \Delta E_k (I_k - \bar{I}), \quad \Delta E = \sum_k \Delta E_k. \quad (5.1)$$

Here ΔE_k and ΔE are modelled load-dependent energy differences in kWh; I_k and $\bar{I}$ have units of g CO₂/kWh; $\Delta C = C_p - C_0$ is in kg CO₂. A positive ΔC means an increase, the opposite sign convention to a reported reduction. The first term is the energy-volume contribution and the second is the time-alignment contribution. Their split depends on the chosen reference intensity, although the net change does not. This is an algebraic decomposition of the scenario outputs, not a causal decomposition of measured power.

Against the selected reference replay on 10 December, B1's 3.3071 kg increase decomposed into approximately 0.1091 kg from energy volume and 3.1979 kg from time alignment. About 96.7% of that net increase therefore lay in the time-alignment term. History-only had a small negative energy-volume contribution but a larger positive time-alignment contribution: consuming slightly less modelled energy did not ensure lower carbon if it occurred in higher-intensity intervals. The decomposition figure uses the saved run pairs, whose reference is stated rather than silently equated with the median-repeat main table.

10 Dec 2025: all seven policies and both Adaptive runs

Positive = increase (opposite sign to the saving figures)



Diamond: exact net change †: zero action Fixed-idle term cancels

Figure 5.11: All eight 10 December policy runs paired with the first baseline, not median headline results. Positive change means increase, $\Delta C = C_p - C_b = -K$. Panel (a) splits $\Delta C = \bar{I}\Delta E/1000 + T$: energy change ΔE (kWh), mean intensity $\bar{I}$ (g CO₂/kWh), and timing residual T (kg CO₂). Panel (b) sums increasing and decreasing intervals. Diamonds give exact net changes; fixed-idle term cancels. † denotes zero-action replay variability. This common-window accounting is not causal attribution.

The fixed 140 kW term is not an explanation for these paired increases: its integral is identical over the common horizon and cancels. Occupancy capping, sharing and placement can change the load-dependent energy integral even when job requests and runtimes are unchanged. On 13 November, for example, B1 saved 1.0936 kg in the capacity model while increasing the requested-node estimate by 2.0919 kg. This disagreement limits a hardware-level interpretation.

Queue effects are another plausible mechanism, but cannot be isolated from repeat variation by these runs alone. On 10 December, B1’s 741 directly delayed jobs had about 16.15 hours of mean additional waiting against the first baseline; the 2,469 jobs without direct delay also had about 9.52 hours more waiting on average. In the same dated History-only run, only two jobs were held, yet their mean absolute predicted-versus-replayed start error was approximately 35.17 hours. These are case-specific diagnostics, not general prediction-error estimates for the policy.

5.5 Completion, timing and retained failures

5.5.1 Completion and runtime errors

Every final replay completed its expected jobs and passed the existing workload and initial-state checks. A later read-only audit nevertheless found at least one logged-runtime error above 60 seconds in 38 of the 55 final replays. Seventeen had no error above that diagnostic line. Completion is therefore reported separately from timing fidelity; the diagnostic was not retroactively used to delete inconvenient dates.

Eight runtime errors exceeded one hour, all in the final extra Adaptive replay on 10 December: seven held carry-in jobs and one evaluation job. Their completion events were processed together after a large gap in the simulation clock. The recorded due times matched logged starts plus configured runtimes within approximately 0.0343 seconds. Crucially, every job had already started before the earliest of these eight events was due. Those particular late completions cannot explain earlier waiting differences. Carbon accounting used configured duration and did not count the entire logged overrun as extra job energy. This narrows the impact of the largest anomaly without proving that all other timing discrepancies are harmless; the clock-gap trigger remains unverified.

The first attempt at that extra Adaptive run had 161 timeouts and was retried under the existing attempt limit. Both attempts’ logs and tables remain archived. The reported denominator is 55 final completed replays, not a claim that the entire process had no failures. Similarly, the bridge comparisons supporting parallel execution on 13 November do not prove every later case equally stable or establish that concurrency caused all variation.

5.5.2 Submission residuals after time alignment

A separate read-only audit of the same 55 final job-result tables measured ϵ_j from Equation 4.6. In 38 replays, at least one job had an absolute residual greater than 60 seconds.

This happens to equal the count for runtime errors above, but the quantities and their affected jobs are different. One concerns submission relative to intended release; the other concerns logged execution duration relative to configured runtime.

Table 5.4: Submission-time residuals in existing final replay outputs. Residuals are measured after each replay’s common median-origin alignment; no simulator was rerun.

Statistic	Population	Value
Replays with any $ \epsilon_j > 60$ s	All final replays	38/55
Largest $ \epsilon_j $	All replay jobs	2,147.69 s
P95 $ \epsilon_j $, 15 November normal admission	1,629 evaluation jobs	2,009.47 s

The largest residual occurred in the first repeated baseline on 15 November and was approximately 35.79 minutes. The evaluation-job P95 in that date’s original baseline was approximately 33.49 minutes, so the issue is not confined to one isolated job. Both exceed the half-hour carbon interval. Residuals vary within a replay, so replacing logged submission with intended release can change a job’s exposure and its overlap with other jobs rather than merely shift the clock origin.

Quantifying the carbon effect of anchoring would require a separate calculation on the same logs, using a consistent alternative start-time definition and a common comparison horizon. Small carbon differences therefore remain conditional on the retained reconstruction. The audit identifies this limitation, but cannot determine whether a strategy’s sign or ranking would reverse.

5.5.3 Negative waiting markers

The same read-only audit found 35 negative logged queue-wait records across 12 of the 55 final replays: 32 evaluation records and three carry-in records. The minimum was -126.010020 seconds. There were also 21 negative reported total-wait records, with a minimum of -83.309414 seconds. These are counts of replay records, not distinct production jobs or negative policy-minus-baseline changes.

The submission marker is an RPC information-log event, not the canonical Slurm accounting submission field. Its ordering relative to allocation can differ, and the inspected logs contain non-monotonic timestamps. The root cause has not been isolated. These values must therefore not be interpreted as physical negative waiting. Equation 4.9 clips negative queue wait only for carbon placement; the frozen waiting summaries retain the raw values. The original completion check did not test this chronology. The maintenance revision now reports completion and timing separately and flags negative values without rewriting the historical evidence. The effects on waiting summaries and carbon rankings have not been recalculated.

5.6 Bounded feedback-extension results

5.6.1 Code behaviour and the functional fixture

All 37 final V2 unit/regression tests passed. They covered feasible alternative candidates, immediate admission, user-budget allocation, spent-delay accounting, repeated submission prevention, unseen-arrival isolation, carbon coverage, the total experiment deadline and refusal to restart a closed experiment. These checks validate the specified behaviours, not the real-world effectiveness of the policy.

In the paired three-job fixture, one job received a hold to 300 seconds, another job from the same user was rejected by the shared budget, and a larger job arrived at 60 seconds. The controller cancelled the first job’s remaining hold near 60.10 seconds. Both runs completed all three jobs, but cancelling the hold did not restore its queue position: that job’s total wait rose from 3.24 to 1,322.36 seconds. It was submitted behind the larger job and then waited for resources. Figure 5.12 makes this difference visible.

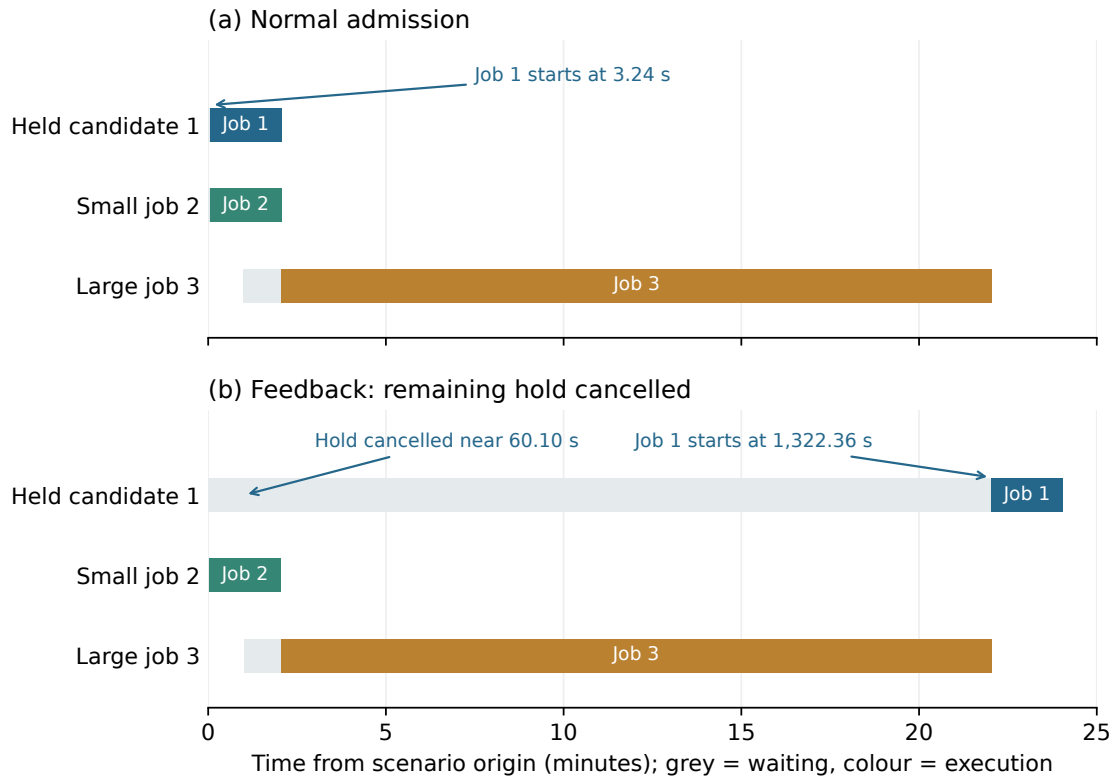


Figure 5.12: Paired three-job functional fixture, not a Stanage performance result. Bars use accounting start and configured runtime. Job 1 was initially held for release at 300 seconds; a larger job arrived at 60 seconds and the controller cancelled the remaining hold near 60.10 seconds. Job 1 nevertheless entered behind that large job and waited 1,322.36 seconds in total, compared with 3.24 seconds under normal admission. Earlier release did not restore the earlier queue position.

This fixture’s capacity carbon proxy fell by about 8.45%, while mean waiting increased by approximately 439 seconds. The designed scenario checks the calculation path; its proxy change is not a Stanage saving and is excluded from the seven-policy result table. The finding concerns queue order: cancelling a hold limits the remaining direct delay, but cannot guarantee recovery of the queue position already lost.

5.6.2 The paired 64-job trace sample

The sample used the same 64 job records and empty initial state in both runs. Both completed all jobs and neither deliberately held any. Of the feedback jobs, 47 were admitted under protection rules and 17 flexible jobs had no feasible hold. Twenty candidate times failed the 5% improvement gate; some short jobs also had budgets too small to reach the next five-minute candidate. Abstention here is an observed response to the fixed rules, not a reason to lower the threshold after seeing the result.

Table 5.5: V2 paired 64-job engineering check. Waiting and runtime errors are in seconds.

Metric	Normal admission	Feedback
Jobs completed	64/64	64/64
Jobs deliberately held	0	0
Mean total wait (s)	1,341.38	1,342.76
P95 total wait (s)	2,362.14	2,411.63
Maximum absolute runtime error (s)	54.66	66.08
Jobs with runtime error above 60 s	0	1

The timing-alignment carbon proxy changed by -0.01321% when expressed as a reduction, while mean wait increased by 1.39 seconds. As no job was actively held, these are not carbon or waiting effects of intentional delay. A single pair cannot separate controller processing from replay variation. The proxy integrates capacity fraction, duration and carbon intensity; it omits the main study’s 140/195 kW conversion and full carry-in queue. It is not a physical per-job footprint or a replacement whole-cage estimate.

One feedback job ran for 279.08 logged seconds against a configured 213 seconds, exceeding the unchanged 60-second timing diagnostic. The warning remains even though completion passed. A separate injected controller-configuration failure triggered the bridge’s fail-open path and all three fixture jobs completed. This covers that startup failure, not every failure mode. Successful simulation attempts still showed the existing exit code 139 after completion, so process shutdown was not reported as clean.

An earlier functional attempt exposed an arrival-grid budget error, which was corrected before the accepted fixture. The first real-record attempt could not resolve simulated nodes under an added network-isolation setting; it was stopped and excluded. Restoring the project’s standard container network allowed the same input to run. These attempts remain recorded, and the combined work finished within about 21 minutes of the single three-hour budget. The extension is complete at this bounded scope; no new full-day experiments are required to interpret these observations.

Chapter 6

Discussion

The five-date comparison gives mixed results. Some policies reduce estimated carbon on several dates, but none meets every frozen stability condition. Interpreting these changes requires attention to their size, the associated waiting and user burden, and variation between replays. The later V2 tests address a separate question: how candidate selection and feedback from the simulated queue behave in the implementation.

6.1 Answers to the research questions

6.1.1 RQ1: Changes in modelled carbon

The policies did not retain the same ordering across dates. B1 illustrates why a count of positive dates is insufficient: its worst increase outweighed four reductions. B2 also had a negative equal-day mean, while Adaptive Balanced and the day/night hybrid had small positive mean percentage reductions. Table 5.3 retains the full numerical comparison.

History-only produced a small positive primary-model mean under several aggregation methods. However, it fell short of the required number of material-positive dates and failed the worst-day condition. The per-job figure divides a shared-queue carbon difference by job count; it cannot be read as an individual’s footprint. Its positive mean also remains sensitive to occupancy representation and time anchoring.

For the tested configurations and workloads, RQ1 has a conditional answer: a lower-intensity admission plan can either reduce or increase estimated carbon once Slurm schedules the shared queue. This agrees with the distinction between ideal temporal opportunity and practically constrained shifting discussed by Sukprasert and colleagues [4]. What matters here is carbon exposure over the resulting execution intervals, not intensity at the selected release timestamp alone.

6.1.2 RQ2: Waiting and user burden

The service question is how much waiting accompanies a carbon change and who bears it. Adaptive’s runtime-proportional bound prevented the direct holds beyond configured runtime

seen under B1. This protection applied to its chosen holds, not to later queueing. Its mean reported additional total wait was lower than B1’s, although the smaller direct-delay budget did not improve every carbon or service outcome.

History-only shows another limitation of the average. On 9 December, a small population mean coexisted with a large share of direct delay concentrated on one user. The cumulative job-hours in Section 5.3.2 add the delays across that user’s jobs; they do not describe one extremely long hold. Limiting each job’s delay and limiting a user’s cumulative burden address different service risks.

User burden here means imposed waiting, not a user carbon account. Wassermann and colleagues show that carbon attribution needs explicit choices about measurement and responsibility for idle resources [26]. The waiting distribution identifies who experiences a service cost, while carbon is assessed for the shared cage scenario.

A carbon reduction need not make service worse for every user. Although direct policy delay is non-negative, total waiting may increase or decrease as jobs compete for resources. Replay variation also contributes to the differences, so each paired change cannot automatically be assigned to deliberate policy action. Positive-only additional waiting reveals later starts that would otherwise be offset by earlier starts elsewhere. Similarly, a favourable population P95 does not mean that every affected job benefited.

6.1.3 RQ3: Information and delay safeguards

Observed runtime helps estimate whether a later interval reduces carbon exposure over a non-preemptive job’s complete execution. A completed baseline schedule also reveals future queue and load information. The given-runtime policies and Oracle reference use these advantages for diagnosis. They therefore need to be distinguished from policies that rely on historical predictors.

Earlier-month quantiles remove some of that hindsight from decision runtime and load. Their performance depends on calibration, support and temporal change. History-only delayed 58 evaluation jobs, of which 44 received direct delays longer than their configured execution runtime. A policy can obey a budget based on a prediction while imposing a large delay relative to the eventual job duration. The representative Oracle and History-only rules also have different thresholds and weights, so their comparison does not isolate the effect of prediction error alone.

The Low-impact rule delayed only three evaluation jobs across the five dates, while the Oracle reference made no deliberate delays. This shows when the implemented rules abstained, rather than proving that the workloads had no useful shifting opportunities. The Oracle rule is not a mathematical upper bound. V2 also identified a selection weakness: ranking before checking all constraints can miss a feasible alternative. The regression test demonstrates this code behaviour, but provides no revised results for the frozen policies.

All methods use a historical carbon series without archived issue-time forecasts. The history-only label applies to runtime and load information; it does not describe a complete online forecast evaluation. Temporal-shifting research shows why forecast uncertainty mat-

ters, but testing an assumed noise process is different from measuring errors in historical forecasts [3]. Runtime, load and carbon information need separate treatment because they come from different sources and have different timing restrictions.

6.1.4 RQ4: Consistency across dates and assumptions

The mean, median, worst date and observed repeat range answer different questions. An equal-day mean weights every selected case equally. The median describes the middle result and is less affected by an extreme date, while the worst date records an adverse outcome that either summary could hide. B1’s median reduction was positive at 0.2205%, even though its mean was negative. Reporting both makes that distribution clear. Standard deviation and interquartile range describe differences across dates; repeat ranges describe variation within a date.

Aggregation changes the question being answered. Adaptive’s positive mean daily percentage and negative normalised and pooled savings in Section 5.3 arise from different weights assigned by daily baseline emissions and job counts. The normalised carbon and waiting means include all evaluation jobs, even those without deliberate delay. Percentiles also require care: averaging daily P95 values does not give a pooled job-level P95, and subtracting two population P95 values does not give the P95 of paired job-level differences.

No policy met the full frozen carbon screen. History-only retained a positive mean direction when any one date was removed, which is useful descriptive evidence. It still failed the material-positive-date and worst-day conditions. The service screen assesses a separate set of costs. Neither screen is a statistical significance test, and every five-date summary retains the negative dates.

Identical-input runs show why repeat variation matters. The 12 no-action cases had no intentional delay to explain their output differences. In particular, Oracle’s added waiting cannot be read as a policy-imposed waiting budget. Baseline and Adaptive repeats reveal some variation, but one run per date for each other policy cannot establish its full distribution.

6.2 What the carbon model supports

The framework compares counterfactual schedules under the same power scenario. The trace records resource allocation, but not application activity or the electricity consumed by individual Stanage jobs. Mapping normalised occupancy to the 55 kW load-dependent range is therefore a modelling assumption. The regular-use estimate of 195 kW came without an occupancy measurement. Placing it at full normalised occupancy defines a scenario endpoint; it cannot establish measured full-load power or validate the model’s slope.

On the common model-completion horizon, the fixed 140 kW term contributes the same integral to each replay and cancels from the paired absolute difference. It remains in the whole-cage percentage denominator. Extra waiting therefore does not automatically add a different fixed idle-energy charge to the policy. For History-only, the sum of paired savings was approximately 0.3922 kg CO₂, equivalent to a pooled reduction of 0.0813% in load-dependent

carbon but only 0.00335% in whole-cage carbon. The execution windows overlap, so this sum is not a continuous five-day emissions account or a basis for annualising savings.

This separation follows the distinction between dynamic and infrastructure components in BLOOM [15], without transferring its component ratios to Stanage.

Disagreement between capacity, node-request and distinct-node views shows sensitivity to the model. History-only’s positive primary-model mean was accompanied by negative mean reductions under both alternatives. All views need to remain visible, rather than selecting the favourable one. An aggregate power series could help calibrate the model and identify which occupancy representation best tracks electrical behaviour. For now, the alternatives show how the conclusion changes with the allocation model; they provide no independent electrical measurements.

Trace-based estimation and HPC power modelling likewise separate resource-to-power validity from scheduling correctness [8, 11]. The three occupancy views are retained rather than selecting the most favourable one.

The regional signal provides an accounting proxy for electricity supply, not a measurement of a job’s causal effect on generation. The literature separates operational accounting from lifecycle and consequential questions [1, 4, 6]. This dissertation reports generation CO₂ within the stated cage-power boundary. Legacy C02e field names do not make it a lifecycle inventory, and the calculation applies no additional PUE multiplier.

In the terminology used by Dodge and colleagues, this is an attributional comparison, while the consequential effect on generation would require marginal-emissions evidence [18]. Agreement among occupancy models does not remove that distinction. It checks an allocation assumption inside the experiment, not the causal response of the electricity system.

6.3 Replay validity and generalisation

Matched inputs and common accounting horizons support comparison, but a completed replay can still have timing errors. Section 5.5 reports these errors. The largest late completions occurred after all jobs had started, so they cannot explain earlier waiting; the carbon model also excludes their full overrun. This narrows the impact of that anomaly. Other timing errors may still matter, and the clock-gap trigger remains unresolved.

The final Adaptive repeat also required a retry after 161 jobs timed out in its first attempt. The failed attempt is retained, while the successful attempt supplies the final result. Thus 55 completed replays is the final comparison denominator, not a statement that every attempt succeeded. The older 8 December pairing has a different validity problem: 24 initially running jobs were restored late in the policy run. Its recorded carbon increase and P95 decrease remain historical audit observations, but neither supports a causal explanation about that policy. This invalid pairing is separate from the five-date results.

The later Slurm simulator work documents variability that supports repeating schedules [28]. Repetition addresses this source of internal uncertainty, but cannot recover missing

production reservations, correct a hardware reconstruction or calibrate a power model. Validation against a physical system requires separate evidence [29].

Submission-time reconstruction is an additional limitation. Section 5.5.2 shows that the residual removed by re-anchoring can exceed one half-hour carbon interval, including at the evaluation-job P95 in one baseline. This is not the same error as a late completion event. Section 5.5.3 also identifies negative waiting markers retained in the original summaries; these are not valid physical waiting intervals. Because jobs can be shifted by different amounts, configured runtime does not by itself preserve logged overlap or establish feasible reconstructed placements. The estimates remain conditional on this accounting convention. The read-only audits do not establish revised savings or rankings, and the maintenance checks do not retroactively validate the original timing.

The five dates were selected for a case study of one cluster; they are not independent random samples of the year. Carry-in work and execution tails can overlap. Each tail drains a finite workload without the full normal arrival stream from later days, which may reduce background competition relative to production. The runs do not establish the direction or size of the resulting policy bias. Assumed flexibility and the fixed basic-priority configuration also limit generalisation. The findings apply to these finite-window cases, without estimating an annual population effect.

The supported completed-work cohort is narrower than the production workload. Failed and cancelled jobs may consume resources without appearing as equivalent replayable jobs here. Runtime caps and carry-in reconstruction also change how source records enter the model. Resource requests and configured runtimes are held fixed between paired policies after this conversion. A complete replay denominator establishes completion of the selected model workload, rather than reproduction of every production job.

6.4 What the feedback extension adds

V2 followed the frozen campaign as a separate engineering extension. It added feasibility checks before candidate ranking and connected a controller to the Slurm simulator event loop. Figure 4.10 shows the arrived jobs and current queue states available to the controller. It can release held jobs when their expected benefit disappears, while keeping spent waiting charged to the user budget. Unlike a static release plan generated before replay, this controller responds to events during the run. The original seven-policy outcomes remain unchanged.

The three-job functional test shows a limit to cancelling a hold. As Figure 5.12 shows, the controller cancelled most of the proposed delay, but a larger job had already entered Slurm. The earlier queue position could not be restored. The fixture tests this interaction between admission and queue order; its carbon proxy change does not measure Stanage performance.

The paired 64-job check in Section 5.6 used records from 18 November. Both runs started empty, and the controller made no deliberate holds. Their small waiting and proxy differences therefore cannot be attributed to shifting jobs. The single pair cannot separate controller

processing from replay variation, and the timing warning remains. This check demonstrates interface operation and rejection behaviour, with no claim about full-day carbon performance.

The 37 saved tests support budget and selection behaviour, not a waiting guarantee from Slurm. V2’s carbon proxy also lacks the main study’s power scaling and remains separate from its five-date tables.

6.5 Practical implications and future work

The project provides an inspectable comparison of admission rules and their service costs. The results support checking carbon exposure over each proposed execution interval, considering cumulative user burden, and allowing a rule to abstain when it cannot meet its constraints. They also show why direct-delay budgets must be assessed alongside realised queue outcomes. These findings inform the design and evaluation of the tested policies; they do not identify one configuration that is best for every workload.

The frozen comparison and bounded V2 checks are complete. Future work could compare re-anchored and log-aligned accounting on a common horizon, investigate inconsistent event timing, or add metered power, issue-time forecasts, user-approved flexibility and continuous background arrivals. These extensions are outside the completed project. The present findings must be read with the stated limitations, without assuming that those further checks have been performed.

Chapter 7

Conclusion

This dissertation evaluated carbon-aware job admission using Stanage accounting records and trace-driven Slurm simulation. The work included auditing the data, reconstructing the initial workload, implementing admission policies and comparing carbon with service outcomes. Slurm supplied the scheduling and resource allocation functions. The contribution lies in the implementation and case-study evaluation built around it. The study neither introduces a new scheduling engine nor measures Stanage electricity use.

The main comparison covered seven policies on five UTC dates, giving 55 final completed replays. The archive supplied timing and resource information but no usable per-job energy, so the carbon results remain scenario estimates. The complete daily tables include negative and no-action outcomes alongside positive results.

For RQ1, carbon changes depended on the policy, date and model. B1’s worst result outweighed four positive dates. History-only produced a small positive equal-day mean reduction of 0.1021% under the primary model, but none of the seven configurations met every frozen stability condition. The cases therefore provide no consistently best policy.

RQ2 showed why direct delay, subsequent queueing and cumulative user burden need separate assessment. Adaptive’s runtime-proportional bound constrained its intended holds. History-only showed how a small mean can conceal delay concentrated on a few users. Carbon and service need to be balanced, although a saving need not worsen every user’s experience.

For RQ3, earlier-month predictors reduced hindsight in runtime and load information, while the carbon input remained retrospective. Budgets based on predictions did not always protect jobs relative to their eventual runtime. Because the tested rules also used different safeguards, their comparison cannot isolate the effect of information quality. No-action results show when those rules abstained; they do not establish that all shifting opportunities were absent.

RQ4 identified sensitivity to occupancy representation, replay variation and accounting choices. Timing audits found material submission residuals and negative waiting markers even in completed replays. Their effects on the frozen estimates have not been recalculated. The retained results apply to re-anchored intervals and finite arrival windows. They cannot be interpreted as an unadjusted production schedule or a measured electricity saving.

The separate V2 extension implemented feasible-first selection, cumulative user budgets and feedback from simulator events. Its functional test showed that cancelling a hold cannot restore an earlier queue position. The 64-job pair made no deliberate holds. These tests provide engineering evidence and remain separate from the five-date policy results.

The completed study provides an inspectable account of how admission decisions, shared-queue waiting and carbon assumptions interact in this Stanage case. The code and manuscript have undergone a final review. Further work could examine timing sensitivity, calibrated power, issue-time forecasts and continuous background arrivals. Those questions extend beyond the completed experiments reported here.

References

- [1] L. Lannelongue, J. Grealey, and M. Inouye, “Green algorithms: Quantifying the carbon footprint of computation,” *Advanced Science*, vol. 8, p. 2100707, 2021. [Online]. Available: <https://doi.org/10.1002/advs.202100707>
- [2] L. F. W. Anthony, B. Kanding, and R. Selvan, “Carbontracker: Tracking and predicting the carbon footprint of training deep learning models,” arXiv:2007.03051, 2020, presented at the ICML Workshop on Challenges in Deploying and Monitoring Machine Learning Systems. [Online]. Available: <https://arxiv.org/abs/2007.03051>
- [3] P. Wiesner, I. Behnke, D. Scheinert, K. Gontarska, and L. Thamsen, “Let’s wait awhile: How temporal workload shifting can reduce carbon emissions in the cloud,” in *Proceedings of the 22nd International Middleware Conference*. ACM, 2021, pp. 260–272. [Online]. Available: <https://arxiv.org/abs/2110.13234>
- [4] T. Sukprasert, A. Souza, N. Bashir, D. Irwin, and P. Shenoy, “On the limitations of carbon-aware temporal and spatial workload shifting in the cloud,” in *Proceedings of the Nineteenth European Conference on Computer Systems*. ACM, 2024, pp. 924–941. [Online]. Available: <https://arxiv.org/abs/2306.06502v2>
- [5] The University of Sheffield, “Stanage specifications,” Sheffield HPC Documentation, https://docs.hpc.shef.ac.uk/en/latest/stanage/cluster_specs.html, accessed 8 September 2026. Current specifications; not a dated inventory of the 2025 trace. [Online]. Available: https://docs.hpc.shef.ac.uk/en/latest/stanage/cluster_specs.html
- [6] U. Gupta, M. Elgamal, G. Hills, G.-Y. Wei, H.-H. S. Lee, D. Brooks, and C.-J. Wu, “ACT: Designing sustainable computer systems with an architectural carbon modeling tool,” in *Proceedings of the 49th Annual International Symposium on Computer Architecture*. ACM, 2022, pp. 784–799. [Online]. Available: <https://doi.org/10.1145/3470496.3527408>
- [7] P. Czarnul, J. Proficz, and A. Krzywaniak, “Energy-aware high-performance computing: Survey of state-of-the-art tools, techniques, and environments,” *Scientific Programming*, vol. 2019, p. 8348791, 2019. [Online]. Available: <https://doi.org/10.1155/2019/8348791>
- [8] K. O’Brien, I. Pietri, R. Reddy, A. Lastovetsky, and R. Sakellariou, “A survey of power and energy predictive models in HPC systems and applications,” *ACM*

- Computing Surveys*, vol. 50, no. 3, pp. 37:1–37:38, 2017. [Online]. Available: <https://doi.org/10.1145/3078811>
- [9] J. You, J.-W. Chung, and M. Chowdhury, “Zeus: Understanding and optimizing GPU energy consumption of DNN training,” in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, 2023, pp. 119–139. [Online]. Available: <https://www.usenix.org/conference/nsdi23/presentation/you>
 - [10] Z. Yang, L. Meng, J.-W. Chung, and M. Chowdhury, “Chasing low-carbon electricity for practical and sustainable DNN training,” Workshop paper, Tackling Climate Change with Machine Learning at ICLR 2023. arXiv:2303.02508v2, 2023, <https://arxiv.org/abs/2303.02508v2>. [Online]. Available: <https://arxiv.org/abs/2303.02508v2>
 - [11] K. West, M. Reid, Y. Elkhatib, and L. Thamsen, “Ichnos: A carbon footprint estimator for scientific workflows,” arXiv:2411.12456v3, 2025, version 3, 6 August 2025; first submitted in 2024 and associated with LOCO 2024. [Online]. Available: <https://arxiv.org/abs/2411.12456v3>
 - [12] F. Antici, A. Bartolini, J. Domke, Z. Kiziltan, and K. Yamamoto, “F-DATA: A Fugaku workload dataset for job-centric predictive modelling in HPC systems,” *Scientific Data*, vol. 12, p. 1321, 2025. [Online]. Available: <https://doi.org/10.1038/s41597-025-05633-1>
 - [13] G. Kamiya and V. C. Coroamă, “Data centre energy use: Critical review of models and results,” EDNA Platform, Technology Collaboration Programme on Energy Efficient End-Use Equipment (4E TCP), Tech. Rep., Mar. 2025, <https://www.iea-4e.org/edna/publications/data-centre-energy-use-critical-review-of-models-and-results/>. [Online]. Available: <https://www.iea-4e.org/edna/publications/data-centre-energy-use-critical-review-of-models-and-results/>
 - [14] B. Li, R. Basu Roy, D. Wang, S. Samsi, V. Gadepally, and D. Tiwari, “Toward sustainable HPC: Carbon footprint estimation and environmental implications of HPC systems,” arXiv:2306.13177v4, 2023, version 4, 19 November 2023; local version used for the methodological comparison. [Online]. Available: <https://arxiv.org/abs/2306.13177v4>
 - [15] A. S. Luccioni, S. Viguier, and A.-L. Ligozat, “Estimating the carbon footprint of BLOOM, a 176b parameter language model,” *Journal of Machine Learning Research*, vol. 24, no. 253, pp. 1–15, 2023. [Online]. Available: <https://www.jmlr.org/papers/v24/23-0069.html>
 - [16] National Energy System Operator, “Carbon intensity API: About and methodology,” Official service documentation, <https://carbonintensity.org.uk/>, accessed 8 September 2026. Reports electricity-generation CO2 intensity, not a complete life-cycle greenhouse-gas inventory. [Online]. Available: <https://carbonintensity.org.uk/>
 - [17] —, “Carbon intensity API for great britain: Regional endpoints and region identifiers,” Official API documentation, <https://api.carbonintensity.org.uk/>, accessed

- 8 September 2026; region 5 is South Yorkshire. [Online]. Available: <https://api.carbonintensity.org.uk/>
- [18] J. Dodge, T. Prewitt, R. Tachet des Combes, E. Odmark, R. Schwartz, E. Strubell, A. S. Luccioni, N. A. Smith, N. DeCario, and W. Buchanan, “Measuring the carbon intensity of AI in cloud instances,” in *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency*. ACM, 2022, pp. 1877–1894. [Online]. Available: <https://doi.org/10.1145/3531146.3533234>
 - [19] K. Xu, D. Sun, H. Tian, J. Zhang, and K. Chen, “GREEN: Carbon-efficient resource scheduling for machine learning clusters,” in *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. USENIX Association, 2025, pp. 999–1014. [Online]. Available: <https://www.usenix.org/conference/nsdi25/presentation/xu-kaiqiang>
 - [20] A. Lechowicz, R. Shenoy, N. Bashir, M. Hajiesmaili, A. Wierman, and C. Delimitrou, “Carbon- and precedence-aware scheduling for data processing clusters,” arXiv:2502.09717, 2025. [Online]. Available: <https://arxiv.org/abs/2502.09717>
 - [21] S. L. Bartholomew, L. Colling, A. Dasgupta, A. J. Greenberg, L. Lannelongue, T. Lestang, C. Martinez-Ortiz, C. Sauzé, A. M. Walker, and A. S. Ward, “CATS: The climate aware task scheduler,” *Journal of Open Source Software*, vol. 10, no. 111, p. 8251, 2025. [Online]. Available: <https://joss.theoj.org/papers/10.21105/joss.08251>
 - [22] CATS contributors, “CATS documentation: Use with schedulers,” Online software documentation, <https://cats.readthedocs.io/en/latest/use-with-schedulers.html>, accessed 10 September 2026. [Online]. Available: <https://cats.readthedocs.io/en/latest/use-with-schedulers.html>
 - [23] SchedMD, “Slurm workload manager: sbatch,” Official command reference, <https://slurm.schedmd.com/sbatch.html>, accessed 8 September 2026; the `–begin` option defers allocation eligibility and does not guarantee an execution start time. [Online]. Available: <https://slurm.schedmd.com/sbatch.html>
 - [24] M. Chadha, J. John, and M. Gerndt, “Extending SLURM for dynamic resource-aware adaptive batch scheduling,” in *2020 IEEE International Conference on High Performance Computing, Data, and Analytics (HiPC)*. IEEE, 2020. [Online]. Available: <https://arxiv.org/abs/2009.08289>
 - [25] M. Luo, S. Zhuang, S. Vengadesan, R. Bhardwaj, J. Chang, E. Friedman, S. Shenker, and I. Stoica, “Starburst: A cost-aware scheduler for hybrid cloud,” in *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. USENIX Association, 2024, pp. 37–57. [Online]. Available: <https://www.usenix.org/conference/atc24/presentation/luo>
 - [26] C. Wassermann, M. Bielert, G. Vanberg, D. Hackenberg, C. Terboven, and M. S. Müller, “Calculating user-centric carbon footprints for HPC,” in *2024 IEEE International*

- Conference on Cluster Computing Workshops*. IEEE, 2024, pp. 26–35. [Online]. Available: <https://publications.rwth-aachen.de/record/997107/files/997107.pdf>
- [27] N. A. Simakov, M. D. Innus, M. D. Jones, R. L. DeLeon, J. P. White, S. M. Gallo, A. K. Patra, and T. R. Furlani, “A Slurm simulator: Implementation and parametric analysis,” in *High Performance Computing Systems. Performance Modeling, Benchmarking, and Simulation*, ser. Lecture Notes in Computer Science. Springer, 2018, vol. 10724, pp. 197–217, pMBS 2017 proceedings; 2018 imprint used as printed in the local chapter. [Online]. Available: https://doi.org/10.1007/978-3-319-72971-8_10
- [28] N. A. Simakov, R. L. DeLeon, Y. Lin, P. S. Hoffmann, and W. R. Mathias, “Developing accurate Slurm simulator,” in *Practice and Experience in Advanced Research Computing (PEARC ’22)*. ACM, 2022, pp. 1–4. [Online]. Available: <https://doi.org/10.1145/3491418.3535178>
- [29] A. Jekanovic, M. D’Amico, and J. Corbalan, “Evaluating SLURM simulator with real-machine SLURM and vice versa,” in *2018 IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*. IEEE, 2018, pp. 72–82. [Online]. Available: <https://doi.org/10.1109/PMBS.2018.8641556>
- [30] N. A. Simakov, R. L. DeLeon, M. D. Innus, M. D. Jones, J. P. White, S. M. Gallo, A. K. Patra, and T. R. Furlani, “Slurm simulator: Improving Slurm scheduler performance on large HPC systems by utilization of multiple controllers and node sharing,” in *Practice and Experience in Advanced Research Computing (PEARC ’18)*. ACM, 2018, pp. 1–8. [Online]. Available: <https://doi.org/10.1145/3219104.3219111>
- [31] SchedMD, “Slurm workload manager: Scheduling configuration guide,” Official software documentation, https://slurm.schedmd.com/sched_config.html, accessed 8 September 2026; describes scheduling semantics, not Stanage’s historical site configuration. [Online]. Available: https://slurm.schedmd.com/sched_config.html
- [32] N. E. Souter, L. Lannelongue, G. Samuel, C. Racey, L. J. Colling, N. Bhagwat, R. Selvan, and C. L. Rae, “Ten recommendations for reducing the carbon footprint of research computing in human neuroimaging,” *Imaging Neuroscience*, 2023. [Online]. Available: https://doi.org/10.1162/imag_a_00043
- [33] SchedMD, “Slurm workload manager: sacct,” Official accounting command documentation, accessed 16 September 2026. <https://slurm.schedmd.com/sacct.html>. [Online]. Available: <https://slurm.schedmd.com/sacct.html>

Appendices

Appendix A

Reproducibility and Evidence Guide

The writing package contains the LaTeX sources, bibliography, figures and aggregate evidence. The implementation and build dependencies are supplied in a separate code-only ZIP. Restricted data, generated workloads and detailed replay outputs remain in the local research archive, so reproducing every result requires inputs beyond either upload package. Earlier writing drafts are preserved but excluded from compilation.

Figure provenance is recorded in the supporting register with the source CSV, generating script, evidence stage and content hash. Existing data files may retain older variable names; captions state the interpretation used in this dissertation. The current campaign is identified as `multiday_20260908`. Its full workload manifests, detailed simulation logs and raw source archive remain in the local research project rather than the shared writing package.

Artefact	Purpose
Workload profile	Source timing, resources, runtime, flexibility and carry-in membership for each simulated job.
Policy decision audit	Candidate information, budgets, selected release and reasons for retaining normal admission.
Replay validation	Complete submissions, starts and completions; timeout and carry-in checks.
Interval carbon output	Common-horizon occupancy, energy and generation-intensity integration under each model.
Daily campaign summary	Matched-date metrics, repetition envelopes and the planned-date completion denominator.
Figure and table register	Mapping from a dissertation visual to its source and empirical status.

Table A.1: Research artefacts and their purposes.

The final main-study snapshot is the completed `multiday_20260908` campaign, closed on 10 September 2026. It contains 55 required final replays and 35 policy–date outcomes. Its original manifest is retained alongside the parallel-execution amendment and carry-in correction record. A separate `policy_v2_20260910` directory contains the bounded extension; it is closed and does not overwrite main-study configurations. The dated source manifests accompanying this manuscript contain full SHA-256 hashes rather than relying on the date or policy label to identify a version.

The local research archive separates final results from retained attempts. The first extra Adaptive attempt on 10 December contains 161 timeouts and remains in its previous-attempt directory. Legacy 8 December outputs are also preserved, although they are not accepted as matched initial-state evidence. Keeping these records allows corrections to be checked against the original files.

The final code archive, `Stanage_Project_Code_Final_20260910.zip`, includes EDA, workload conversion, admission, carbon accounting, tests, configuration, modified Slurm source and the independent V2 implementation. Its `submission_manifest.json` records hashes, entry points and exclusions. The separate `maintenance_revision.json` identifies post-study reporting fixes; the earlier package remains preserved locally. These fixes changed no policy algorithm, carbon arithmetic or simulator C source. Notebook code cells are retained without outputs. Papers, slides, Markdown, raw data and experiment results are excluded, while required licences and embedded command-line resources remain. Build declarations omit upstream manuals and the upstream test-suite build.

Entry points are `stanage_eda/scripts/analyse_stanage.py` for EDA and, under `simulator/slurm_policy_experiments/`, `build_exact_24h_workload.py`, `generate_policy_scenario.py` and `calculate_carbon.py` for preparation, admission and accounting. Regenerate historical predictors with `build_history_quantile_model.py` and obtain carbon data with `download_neso_carbon.py`. The authorised Stanage archive is required separately; examples do not replace it.

The package includes Python dependencies and a test entry point. After extraction outside the project, the earlier package passed 146 tests and two subtests. Its Slurm source also passed configuration, compilation and installation on Linux with networking disabled; no replay was run. The separately extracted final maintenance package passed 159 tests and five subtests, including checks for negative timing, raw-wait preservation, exit recording and read-only auditing. Historical launchers retain Colima context choices and a local archive path that need configuration on another host. These tests and builds do not establish one-command reproduction or bit-identical schedules elsewhere.

The residual and negative-wait audits read the 55 final tables without replacing historical metrics; their aggregate evidence accompanies this manuscript. The packaged `tools/maintenance/audit_submission_timing.py` accepts an external campaign and requires a new output directory outside it. Both packages exclude raw records, real user identifiers, private configuration and the EVRP reference PDF.

Appendix B

Complete five-date results

The following tables contain all seven policies on all five dates. They preserve the same evaluation cohort within each date and retain negative and zero-action results. Primary carbon uses median values from three baseline replays and two Adaptive replays; the other policies have one replay per date. Paired waiting is calculated for matching evaluation-job IDs, then summarised across replay pairs. A dagger identifies audited zero-action cases with executable inputs identical to baseline: their differences are replay variation rather than an intentional-delay effect.

Carbon values describe the modelled shared queue, not per-job meter readings. Dividing a shared saving by the evaluation-job count does not uniquely attribute carbon to those jobs. Alternative occupancy percentages test sensitivity to the model; they are not independent measurements. Carry-in and execution tails can overlap between dates, so these tables cannot be read as a continuous five-day or annual emissions account.

Table B.1 provides absolute and percentage carbon changes, including the primary whole-cage percentage. The service table separates direct hold, actual added total waiting and concentration of user burden. A P95 of job-paired wait differences is not the difference between two population P95 values. Companion CSVs retain full precision and the extra model/scope fields; the source register is `supporting/final_results_source_manifest_20260910.csv`.

Appendix B. Complete five-date results

Table B.1: All 35 daily carbon results (2025). $K_C = C_{b,C} - C_{p,C}$ is primary cluster saving (kg CO₂); $g_C = 1000K_C/N$ (g CO₂ per evaluation job). R_C, R_R, R_A are load-related saving percentages for capacity-weighted, node-request upper and distinct allocated-node proxies; $R_{C,w}$ is whole-cluster primary saving percentage. Negative means increase. † marks zero-action replay variability. Policy abbreviations follow the results chapter.

Date	Policy	N (jobs)	K_C (kg)	g_C (g/job)	R_C (%)	$R_{C,w}$ (%)	R_R (%)	R_A (%)
11/13	B1	5806	+1.0936	+0.1884	+1.3835	+0.0603	-0.7234	+1.5710
11/13	B2	5806	+0.4482	+0.0772	+0.5670	+0.0247	-0.2529	-0.4387
11/13	AD	5806	+0.5773	+0.0994	+0.7304	+0.0318	-0.3090	+2.6291
11/13	HY	5806	+0.6269	+0.1080	+0.7931	+0.0345	-0.2946	-0.9804
11/13	HI	5806	+0.2177	+0.0375	+0.2754	+0.0120	-0.2737	-1.7535
11/13	OR†	5806	-0.0019	-0.0003	-0.0024	-0.0001	+0.0016	-0.9431
11/13	LI	5806	+0.0144	+0.0025	+0.0182	+0.0008	-0.0150	-0.3784
11/15	B1	1629	+0.0654	+0.0401	+0.0930	+0.0029	-0.0412	-1.9977
11/15	B2†	1629	+0.0140	+0.0086	+0.0199	+0.0006	-0.0059	-1.0220
11/15	AD	1629	+0.0014	+0.0008	+0.0019	+0.0001	-0.0092	-0.1168
11/15	HY	1629	+0.0613	+0.0377	+0.0873	+0.0027	-0.0378	-1.2631
11/15	HI†	1629	+0.0072	+0.0044	+0.0103	+0.0003	-0.0121	-1.0736
11/15	OR†	1629	+0.0008	+0.0005	+0.0011	+0.0000	+0.0019	-1.2413
11/15	LI†	1629	-0.0004	-0.0003	-0.0006	-0.0000	+0.0007	-0.4629
11/18	B1	5646	+0.6000	+0.1063	+0.5534	+0.0198	-0.0827	+0.7304
11/18	B2	5646	-0.0278	-0.0049	-0.0256	-0.0009	-0.0110	-0.6907
11/18	AD	5646	+0.1838	+0.0326	+0.1696	+0.0061	+0.0011	-1.2976
11/18	HY	5646	+0.1906	+0.0338	+0.1758	+0.0063	+0.0190	-1.3920
11/18	HI	5646	+0.2622	+0.0464	+0.2418	+0.0087	+0.0038	-0.9558
11/18	OR†	5646	-0.0206	-0.0036	-0.0190	-0.0007	-0.0057	-0.6663
11/18	LI	5646	+0.0169	+0.0030	+0.0156	+0.0006	+0.0107	-1.9201

Continued on next page

Appendix B. Complete five-date results

Table B.1 continued

Date	Policy	N (jobs)	K_C (kg)	g_C (g/job)	R_C (%)	$R_{C,w}$ (%)	R_R (%)	R_A (%)
12/09	B1	3113	+0.2190	+0.0704	+0.2205	+0.0107	-0.6278	-1.0329
12/09	B2 [†]	3113	+0.1012	+0.0325	+0.1019	+0.0049	+0.1894	+1.1534
12/09	AD	3113	+0.3880	+0.1246	+0.3905	+0.0189	-0.0115	-0.9611
12/09	HY	3113	+0.1812	+0.0582	+0.1824	+0.0088	-0.0156	+0.2473
12/09	HI	3113	+0.2840	+0.0912	+0.2859	+0.0138	+0.0574	-1.7321
12/09	OR [†]	3113	-0.1130	-0.0363	-0.1137	-0.0055	+0.0110	-0.4866
12/09	LI [†]	3113	+0.0328	+0.0105	+0.0330	+0.0016	+0.0786	+1.2482
12/10	B1	3210	-3.3071	-1.0302	-2.6427	-0.1284	-4.9396	-1.0636
12/10	B2 [†]	3210	-1.6879	-0.5258	-1.3488	-0.0655	-0.3644	-0.8082
12/10	AD	3210	-1.2113	-0.3774	-0.9680	-0.0470	-0.3399	-0.6495
12/10	HY	3210	-0.8236	-0.2566	-0.6582	-0.0320	-0.4685	+4.4877
12/10	HI	3210	-0.3789	-0.1180	-0.3028	-0.0147	-0.2004	+1.1168
12/10	OR [†]	3210	-1.0098	-0.3146	-0.8069	-0.0392	-0.1389	+1.5631
12/10	LI [†]	3210	-1.7174	-0.5350	-1.3723	-0.0667	-0.3104	-0.9921

Values use four decimal places; signed zero retains the unrounded direction. The companion CSV preserves source precision for all six model–scope combinations, including whole-cluster R and A values and model-specific repeat ranges. Frozen CSV sources and SHA-256 hashes for all new result assets are recorded in `supporting/final_results_source_manifest_20260910.csv`.

Appendix B. Complete five-date results

Table B.2: All 35 daily wait and burden results (2025). n_D counts directly held jobs; $\bar{D}$ is mean hold. $\overline{\Delta W}$ and $P95(\Delta W)$ summarize paired added total wait; ΔW_0 is the mean among jobs not directly held. U_D, U_+ are the largest user's shares of total hold and positive added wait. Times are minutes per job in the stated cohort. $\dagger$ marks zero-action replay variability.

Date	Policy	n_D (jobs)	$\bar{D}$ (min)	$\overline{\Delta W}$ (min)	$P95(\Delta W)$ (min)	ΔW_0 (min)	U_D (%)	U_+ (%)
11/13	B1	1026	36.35	+35.26	+224.07	-1.38	50.6	50.3
11/13	B2	705	22.02	+21.00	+239.37	-1.22	57.7	57.2
11/13	AD	470	12.51	+11.62	+99.88	-1.06	77.1	75.8
11/13	HY	860	31.70	+30.63	+239.33	-1.26	62.1	61.7
11/13	HI	10	0.32	+0.40	+1.09	+0.05	63.7	41.9
11/13	OR †	0	0.00	-0.08	+0.21	-0.08	–	62.8
11/13	LI	2	0.02	+0.04	+0.88	+0.02	50.0	65.3
11/15	B1	324	29.12	+26.45	+213.65	-6.04	52.1	47.6
11/15	B2 †	0	0.00	-0.60	+1.21	-0.60	–	31.3
11/15	AD	73	2.00	+3.10	+43.76	-2.42	73.4	61.3
11/15	HY	252	21.80	+19.36	+187.96	-5.59	43.1	39.1
11/15	HI †	0	0.00	+0.72	+9.14	+0.72	–	51.5
11/15	OR †	0	0.00	+0.00	+3.71	+0.00	–	46.6
11/15	LI †	0	0.00	+0.56	+5.79	+0.56	–	48.5
11/18	B1	1423	40.31	+39.57	+221.02	-0.23	61.1	62.1
11/18	B2	260	1.39	+1.07	+3.77	-0.20	51.5	52.7
11/18	AD	64	1.58	+1.11	+1.87	-0.17	37.5	35.3
11/18	HY	586	13.62	+12.79	+194.29	-0.31	67.1	69.3
11/18	HI	5	0.16	+0.15	+3.54	+0.05	50.1	41.8
11/18	OR †	0	0.00	+0.64	+5.63	+0.64	–	50.6
11/18	LI	1	0.01	+0.28	+2.30	+0.27	100.0	52.0

Continued on next page

Appendix B. Complete five-date results

Table B.2 continued

Date	Policy	n_D (jobs)	$\bar{D}$ (min)	$\overline{\Delta W}$ (min)	$P95(\Delta W)$ (min)	ΔW_0 (min)	U_D (%)	U_+ (%)
12/09	B1	820	41.67	+40.92	+363.28	-6.71	26.9	37.9
12/09	B2 [†]	0	0.00	-5.81	+27.63	-5.81	–	26.0
12/09	AD	176	7.54	+28.05	+85.74	+20.68	31.7	29.0
12/09	HY	175	8.73	+23.08	+93.00	+22.70	43.8	39.3
12/09	HI	41	2.00	+13.97	+57.14	+10.33	86.8	32.4
12/09	OR [†]	0	0.00	-7.07	+12.25	-7.07	–	21.7
12/09	LI [†]	0	0.00	+0.66	+44.13	+0.66	–	33.7
12/10	B1	741	30.91	+663.10	+1223.42	+571.36	54.9	48.3
12/10	B2 [†]	0	0.00	+78.60	+154.31	+78.60	–	48.8
12/10	AD	61	1.15	+52.14	+123.13	+51.75	26.7	53.9
12/10	HY	81	4.04	+69.43	+272.14	+65.00	20.8	39.8
12/10	HI	2	0.13	+9.61	+73.42	+9.60	52.3	34.6
12/10	OR [†]	0	0.00	+50.70	+92.20	+50.70	–	54.5
12/10	LI [†]	0	0.00	+85.85	+159.27	+85.85	–	52.7

$\bar{D}$ and $\overline{\Delta W}$ include all evaluation jobs; P95 is a percentile of paired differences. – means undefined hold concentration because total hold is zero. The companion CSV retains source seconds, P99, positive wait, delayed-beyond-runtime counts and maximum-user burden aggregates. No raw user identifiers are included.

Appendix C

Acknowledgement of generative AI assistance

I used GPT-5.6-Sol and GPT-6-Astra through tools including OpenAI Codex and Cursor. They helped with preparing and revising explanatory text from project notes, supervision feedback and recorded results, as well as language editing, translation, summaries, literature searches, discussion of research and comparison methods, and $\text{\LaTeX}$ equation formatting.

AI also assisted with writing, revising and debugging parts of the research code, running tests, and preparing plotting and explanatory-diagram code and layout. This involved repeated instructions and revisions against the project requirements. The reported findings come from job data and saved program outputs; technical claims refer to original papers and official documentation, not AI responses.

I acknowledge this assistance rather than presenting the relevant work as wholly unaided. Review included code tests, source checks and comparisons with saved results. The dissertation reports unresolved limitations. I take responsibility for the submitted work, its attribution and interpretation. Conversation histories, revision records and test logs document the assistance provided.

Appendix D

Abbreviations

HPC	High-performance computing
CI	Carbon intensity
EDA	Exploratory data analysis
NESO	National Energy System Operator
PUE	Power usage effectiveness
UTC	Coordinated Universal Time
CPU / GPU	Central / graphics processing unit